2026

CXI AFRICA TRAINING HANDBOOK-CHIETA DATALAB

Prof Pius A.Owolawi
CXI AFRICA
8/24/2026

Table of Contents

CHIETA DATA LAB TRAINING PROGRAMME 4
Six-Month Study Plan & Programme Handbook 4
1. Summary Introduction 4
2. Reasoning and Design Basis 5
2.1 What each source document contributes 5
2.2 The four conflicts between the sources, and how each is resolved 6
2.3 The pedagogical model 8
2.4 Why the assessment model is what it is 8
2.5 Compliance, ethics and data safety 9
3. The Case-Study Spine and the Competency Model 9
3.1 The spine: one artefact, twenty-four weeks 9
3.2 The competency framework 10
3.3 The badge ladder 11
4. Table of Contents 11
FRONT MATTER — pages i–viii (numbered 1–8 in this outline) 11
PART A — PROGRAMME ARCHITECTURE — pages 9–24 12
PART B — MONTH 1: FOUNDATIONS OF COMPUTING & PROGRAMMING — pages 25–38 12
PART C — MONTH 2: FRONTEND ENGINEERING WITH NEXT.JS — pages 39–52 12
CONSOLIDATION WEEK 1 — page 53 13
PART D — MONTH 3: BACKEND ENGINEERING & DATA PERSISTENCE — pages 54–67 13
PART E — MONTH 4: DATA ENGINEERING, STORAGE & CONTAINERS — pages 68–80 13
PART F — MONTH 5: ANALYTICS, MACHINE LEARNING & AZURE — pages 81–94 14
PART G — MONTH 6: QUALITY, DEVSECOPS, CAPSTONE & HANDOVER — pages 95–108 14
PART H — THE ASSESSMENT SYSTEM — pages 109–115 14
PART I — FACILITATOR PLAYBOOK — pages 116–122 15
PART J — LEARNER SUPPORT, PROGRESSION & PROFESSIONAL PRACTICE — pages 123–128 15
PART K — APPENDICES — pages 129–147 15
5. Detailed Outline — Front Matter (pages 1–8) 16
6. Detailed Outline — Part A: Programme Architecture (pages 9–24) 16
A.1–A.3 Purpose, learner profile, diagnostic (pages 9–11) 17

A.4–A.5 Competency framework and badges (pages 12–14) 17
A.6–A.7 The spine and the map (pages 15–17) 17
A.8–A.10 Delivery model (pages 18–20) 17
A.11–A.14 Environment, hardware, access, data safety (pages 21–24) 18
7. Detailed Outline — Part B: Month 1, Foundations (pages 25–38) 19
B.0 Month opener (page 25) 19
B.1 Learning objectives (page 26) 19
B.2–B.8 Weekly breakdown (pages 27–33) 20
B.9 Case Study CS-1 — *The Records* (pages 34–35) 21
B.10 Assessments and the Month 1 gate (page 36) 22
B.11 Resources and skills checklist (page 37) 22
B.12 Facilitator notes (page 38) 23
Expected outcomes at the end of Month 1 23
8. Detailed Outline — Part C: Month 2, Frontend Engineering (pages 39–52) 23
C.0 Month opener and the link back to CS-1 (page 39) 23
C.1 Learning objectives (page 40) 24
C.2–C.9 Weekly breakdown (pages 41–48) 24
C.10 Case Study CS-2 — *The Shell* (pages 49–50) 25
C.11 Assessment, gate and badge (page 51) 26
C.12 Resources and facilitator notes (page 52) 26
Expected outcomes at the end of Month 2 27
9. Consolidation Week 1 (page 53) 27
10. Detailed Outline — Part D: Month 3, Backend & Data Persistence (pages 54–67) 27
D.0 Month opener (page 54) 28
D.1 Learning objectives (page 55) 28
D.2–D.10 Weekly breakdown (pages 56–64) 28
D.11 Case Study CS-3 — *The Spine* (pages 65–66) 29
D.12 Assessment, gate, badge, resources, facilitator notes (page 67) 30
Expected outcomes at the end of Month 3 30
11. Detailed Outline — Part E: Month 4, Data Engineering, Storage & Containers (pages 68–80) 31
E.0 Month opener (page 68) 31
E.1 Learning objectives (page 69) 31
E.2–E.9 Weekly breakdown (pages 70–77) 31

E.10 Case Study CS-4 — *The Pipeline* (pages 78–79) 32
E.11 Assessment, gate, badge, resources, facilitator notes (page 80) 33
Expected outcomes at the end of Month 4 34
12. Detailed Outline — Part F: Month 5, Analytics, ML & Azure (pages 81–94) 34
F.0 Month opener (page 81) 34
F.1 Learning objectives (page 82) 34
F.2–F.11 Weekly breakdown (pages 83–91) 34
F.11 Case Study CS-5 — *The Insight* (pages 92–93) 36
F.12 Assessment, gate, badge, resources, facilitator notes (page 94) 37
Expected outcomes at the end of Month 5 37
13. Detailed Outline — Part G: Month 6, Quality, DevSecOps, Capstone & Handover (pages 95–108) 37
G.0 Month opener (page 95) 38
G.1 Learning objectives (page 96) 38
G.2–G.9 Weekly breakdown (pages 97–105) 38
G.10 Case Study CS-6 — *The Product* (pages 106–107) 39
G.11 Final assessment, badge and exit (page 108) 40
Expected outcomes at the end of Month 6 41
14. Detailed Outline — Parts H, I, J (pages 109–128) 41
Part H — The Assessment System (pages 109–115) 41
Part I — Facilitator Playbook (pages 116–122) 42
Part J — Learner Support, Progression & Professional Practice (pages 123–128) 42
15. Detailed Outline — Part K: Appendices (pages 129–147) 43
16. Two Fully Written Sample Pages 44
★ SAMPLE PAGE 1 — Handbook page 30 44
B.5 Week 3, Session 1 — Functions: Teaching a Program to Remember How 44
★ SAMPLE PAGE 2 — Handbook page 86 47
F.5 Week 23, Session 2 — Training and Evaluating the Dropout-Risk Model 47
17. Production, Editing and Format Notes 50
17.1 What has been delivered and what remains 50
17.2 Editing the master file 50
17.3 Conversion 51
17.4 The objectives this handbook must keep meeting 51

CHIETA DATA LAB TRAINING PROGRAMME

Six-Month Study Plan & Programme Handbook

- **Delivered by:** CXi Africa (Centre for Digital Transformation and Innovation Africa)
- **Client / Sponsor:** CHIETA — Chemical Industries Education & Training Authority
- **Programme:** CHIETA Data Lab — Data Science Solution & Prototype Development
- **Cohort type:** Absolute beginners (no prior programming experience assumed)
- **Duration:** 26 calendar weeks (24 teaching weeks + 2 consolidation weeks)

---

1. Summary Introduction

This document is the master blueprint for a six-month training programme that takes a person who has never written a line of code and leaves them, twenty-six weeks later, able to build a working Data Lab application end to end — frontend, backend, database, object storage, a data pipeline, a machine-learning prediction feature, tests, a CI/CD pipeline, and a cloud deployment — and able to explain every decision they made along the way.

That is an unusually ambitious claim for a beginner programme, and it is worth stating up front why it is achievable here when it usually is not. Three things make the difference.

First, there is only one project. Most beginner curricula teach a language using throwaway exercises, then teach a framework using a different throwaway exercise, then ask the learner to invent a capstone from nothing in the final month. The gap between "I completed forty tutorials" and "I can build a system" is where most learners are lost. This programme removes that gap by having learners build *one* artefact continuously from Week 1 to Week 24. Every exercise, every lab, every assessment is a contribution to the same growing codebase — the **CHIETA Skills Data Lab (Mini)**. In Week 3 a learner writes a Python function that calculates a completion rate; in Week 18 that same calculation is a feature in a machine-learning model; in Week 22 it is covered by an automated test that runs on every pull request. Nothing is thrown away. This is the *case-study spine*, and it is described in full in Section 3.

Second, the sequence respects how beginners actually build competence. Concepts are introduced in the order that makes each one *necessary* rather than arbitrary. Learners meet Python before TypeScript because Python's syntax stays out of the way while the harder ideas — variables, control flow, decomposition — are being installed. They meet plain JavaScript before TypeScript so that TypeScript feels like a solution to a problem they have personally experienced rather than extra ceremony. They meet a local file-write before object storage, and a single container before an orchestrated stack. Every new tool arrives immediately after the learner has felt the pain that the tool exists to remove. This is deliberate and it is the single most important pedagogical decision in the programme.

Third, the pacing is honest. The three source documents that feed this handbook disagree — sharply — about how many hours a week this programme demands. That disagreement is not glossed over here; it is resolved in Section 2.3 with a two-track delivery model that guarantees a defensible outcome at fifteen hours per week and offers a stretch path at twenty-five. A

curriculum that quietly assumes twenty-five hours and is delivered to learners who have twelve is a curriculum that fails in Month 4 with no warning.

The handbook that follows is written for three audiences at once. **Learners** get a clear map of what they will be able to do at the end of each month, what is expected of them each week, and what "good" looks like before they are assessed on it. **Facilitators** get session-level teaching plans, the common misconceptions to watch for, checkpoint criteria, and remediation routes for learners who fall behind. **CHIETA and CXi Africa programme management** get a competency framework, an assessment architecture that produces auditable evidence, alignment to the CHIETA Data Lab Project Handbook (Volumes I–VIII, FR-1 to FR-40), and a handover package that survives the departure of any individual.

What follows is presented in the sequence: reasoning and design basis first (Section 2), the case-study spine and competency model (Section 3), then the full table of contents (Section 4), then the page-by-page outline of all 118 pages (Sections 5–15), then two fully written sample pages demonstrating the expected depth and format (Section 16), then production and editing notes (Section 17).

---

2. Reasoning and Design Basis

Before any framework, template, or study plan is presented, this section sets out the evidence the design rests on, what each source document contributes, where those documents conflict, and how each conflict has been resolved. Facilitators adapting this programme should read this section first; it explains *why* the structure is what it is, and therefore which parts can safely be changed and which cannot.

2.1 What each source document contributes

Three documents were supplied as the foundation for this programme. They are not redundant — each solves a different problem, and the handbook needs all three.

Source document	What it is strong at	What it lacks on its own	How it is used here
Lesson_plan.docx — 8 phases across 30 weeks, module-structured, platform-oriented	Breadth and professional completeness. It is the only source that covers Polars/DuckDB, interactive notebooks, collaboration features, data governance and lineage, responsible/explainable AI, and reflective professional practice. Its module framing ("Module 1: Python Programming") is the cleanest unit for a handbook.	It has no weekly cadence, no deliverables, no assessment mechanics, and assumes 30 weeks, which does not fit a six-month contract. Its "Education Standards" section is an unfilled placeholder.	Supplies the **module taxonomy and topic coverage floor**. Every module in this handbook maps back to a Lesson Plan module. Its Phase 6 (Collaboration) is dissolved into the building months rather than run as a standalone block. Its Education Standards

			placeholder is filled in Section 5.4 with an NQF/SETA-referenced competency framework.
CHIETA_Data_Lab_6_Month_Beginner_Course.docx — 24 weeks, 6 months, explicit stack	The stack decision (Next.js 14 App Router, FastAPI, PostgreSQL, MinIO, Airflow, scikit-learn/MLflow, Docker, Azure), the mapping to Project Handbook Volumes I–VIII and FR-1 to FR-40, month-end mini-projects, skills checklists, the badge ladder, and a genuinely beginner-appropriate entry point.	Weeks are given as bullet lists, not teachable sessions. No rubrics, no remediation path, no facilitator scripts, no data dictionary, no risk register. The 24–26 hr/week budget is asserted without a fallback.	Supplies the **spine of the six-month sequence, the technology stack, the month-end project titles, the badge ladder, and the FR-coverage obligation**. This handbook's Month 1–6 structure is a direct expansion of this document.
CHIETA_DataLab_Intern_Training_Plan__1_.docx (Lethabo Molefe) — 4 phases, 24 weeks, delivery-oriented	Everything operational: the weekly rhythm (Mon stand-up / Tue taught / Wed deep work / Thu taught + review / Fri wrap-up), the four-level support escalation ladder, the assessment weighting model, the contribution log, phase-end checkpoints with explicit consequences, module ownership from Month 4, the five intern roles, the full software install list, and the monthly one-on-one.	Its curriculum is narrower — Python then TypeScript/React with SQL, but no ETL, no MinIO, no ML, no Docker, no cloud, no CI/CD. It also assumes learners arrive with *some* coding ability, which contradicts the brief.	Supplies the **entire delivery and support architecture**: weekly rhythm, escalation ladder, checkpoints, assessment weights, contribution log, role assignment, environment setup. Its curriculum is treated as the *minimum floor* for Months 1–3 and is extended, not replaced. Its "interns already code a little" assumption is explicitly reversed — see 2.2.

2.2 The four conflicts between the sources, and how each is resolved

These are real contradictions. Any handbook that merges the three documents without addressing them will fail in delivery, so each is resolved here with a stated rationale.

Conflict 1 — Entry level. The Intern Training Plan states interns can "write basic code" and have "simple CRUD apps" behind them. The Beginner Course states the audience is "absolute beginners (no prior programming)." The commissioning brief for this handbook is unambiguous: zero experience.

Resolution: The brief wins. The programme is designed for a learner who has never opened a terminal. This has three concrete consequences. (a) Month 1 is expanded from the Intern Plan's four weeks of Python into **five weeks of foundations**, with Week 0 added as a pre-programme onboarding week covering hardware, accounts, file systems, and keyboard/terminal literacy before any code is written. (b) Every abstraction is introduced with a **physical or paper analogue before the code** — learners trace loops on paper before running them, draw the request/response cycle before opening DevTools, and sketch tables before writing SQL. (c) A **diagnostic** is run in Week 0 (Appendix D) so that any learner who *does* arrive with prior experience is routed to the Extension Track rather than being bored for five weeks.

Conflict 2 — Weekly time budget. The Intern Plan budgets ≈12 hrs/week (288 hours total). The Beginner Course budgets 24–26 hrs/week (≈600 hours total). That is a 2× difference in total contact, and the Beginner Course curriculum cannot be delivered in 288 hours.

Resolution: A **two-track model**. The **Core Track** is 15 hrs/week (≈390 hours across 26 weeks) and is what every learner must complete; it delivers the full capstone at a defensible standard. The **Extension Track** adds 8–10 hrs/week of stretch work (Superset, Spark, full AKS/Helm, OpenMetadata, Polars/DuckDB, advanced testing) for learners with the capacity, and is the route to the top badge. All assessment, all checkpoints and all certification thresholds are set against the Core Track only. Nobody fails for not doing extension work. Section 5.7 gives the full hour-by-hour breakdown of both tracks.

Conflict 3 — Programme length. The Lesson Plan runs 30 weeks across 8 phases. The other two run 24 weeks. Six months is roughly 26 weeks.

Resolution: 26 weeks — 24 teaching weeks plus 2 consolidation weeks (a mid-programme consolidation week after Week 12, and a pre-capstone consolidation week before Week 24). The Lesson Plan's 30 weeks are compressed by (a) dissolving its Phase 6 (Collaboration) into Months 3–6 as continuous practice rather than a four-week block — collaboration is a habit, not a topic — and (b) folding its Phase 8 (Capstone) into the existing Month 6. The consolidation weeks are non-negotiable: they are where learners who have fallen behind catch up without the cohort stalling, and where those who are ahead do extension work. A programme with no slack becomes a programme that abandons its weakest learners in Month 4.

Conflict 4 — Teaching language order. The Intern Plan teaches Python for a month, then moves entirely to TypeScript/React for two months, then builds. The Beginner Course interleaves Python, JavaScript, TypeScript and SQL from Month 1 and returns to Python for the backend in Month 3.

Resolution: A **Python-first spine with early, shallow exposure to the web**. Python is the teaching language for programming *principles* (Weeks 1–5) because syntax overhead is lowest. HTML/CSS and a small amount of JavaScript appear in Week 4–5 so learners see something visual early — motivation matters enormously for beginners in Month 1 — but the web stack is not taught in depth until Month 2. Python then returns as the *backend* language in Month 3, which is the moment learners discover

that the language they learned first is the language the system actually runs on. That return is deliberately staged as a confidence event.

2.3 The pedagogical model

Five principles govern every session in this handbook. They are stated here so that facilitators writing new material stay consistent with the existing material.

Principle 1 — Ship something visible every single week. Every week ends with an artefact the learner can see, run or click: a script that produces output, a page that renders, an endpoint that responds, a container that starts, a chart that draws. Invisible progress is indistinguishable from no progress, and beginners who cannot see progress quit. This is inherited directly from the Beginner Course facilitator notes and is treated as a hard constraint on lab design.

Principle 2 — Concrete before abstract, always. The class writes the repetitive version before the loop, the copy-pasted version before the function, the untyped version before the interface, the manual deploy before the pipeline. The abstraction is then introduced as *relief*. Teaching the abstraction first produces learners who can recite the definition and cannot apply it.

Principle 3 — Break it on purpose. Every lab includes a deliberate-failure step: delete a semicolon, drop a required field, stop the database container, pass the wrong type. Learners must read the resulting error out loud and say what it means before fixing it. Beginners' single greatest handicap is fear of error messages; this is the direct cure, and it comes from the Beginner Course's recommended practice order (Appendix B of that document).

Principle 4 — The five-step topic loop. Adapted from the Beginner Course's Appendix B, every new topic follows: **(1)** 15–20 minute concept demo → **(2)** type-along lab with the facilitator → **(3)** break it deliberately, then fix it → **(4)** rebuild the core of it from memory without notes → **(5)** state, in one sentence, how it connects to a CHIETA user story. Step 5 is what converts a technical exercise into a professional one and it is skipped far too often.

Principle 5 — Independence is trained, not assumed. The support ladder from the Intern Training Plan is adopted verbatim in structure: self-help first, peer help after 30 minutes, mentor after 2 hours, session escalation for architectural questions. The two-hour rule is a *ceiling*, not a target — learners are not being taught to struggle silently, they are being taught to struggle productively and then to ask a well-formed question. The four-part help request format (what I tried, what I expected, what happened, what I've already tried) is assessed as a graded collaboration competency, not merely encouraged.

2.4 Why the assessment model is what it is

There are no written examinations in this programme. That is a deliberate choice inherited from the Intern Training Plan and reinforced by the Beginner Course's facilitator note to "grade working demos and understanding, not memorised jargon."

The reasoning: the competency being certified is *the ability to build and explain a working system*. A written exam measures recall of terminology, which correlates weakly with that competency and correlates strongly with prior educational advantage — a serious equity problem in a SETA-funded programme whose purpose is to widen access. Assessment is therefore built on four evidence types that are all directly observable: **merged code**, **completed weekly**

deliverables, **live demonstrations**, and **spoken/written explanation of one's own decisions**. The final assessment is a capstone demo plus a short technical interview, exactly as the Beginner Course specifies.

The weighting model is adapted from the Intern Training Plan with two changes, both justified: **weekly deliverables are raised from 20% to 25%** because in a true beginner cohort the weekly cadence is the main driver of completion, and **a 5% weighting for the entry diagnostic and learning-plan artefact is removed entirely** so that no learner is penalised for their starting point. Full rubrics are in Part H and Appendix E.

2.5 Compliance, ethics and data safety

Two constraints apply throughout and are not optional.

No real CHIETA personal data is used in training, at any point, in any environment. All datasets in this programme are synthetic, generated for the purpose, and shipped with the courseware (Appendix A). This is a direct requirement in the Beginner Course facilitator notes and it aligns with South Africa's Protection of Personal Information Act (POPIA). Learner laptops are not secure processing environments; treating them as such would create real regulatory exposure for CHIETA. The synthetic datasets are designed to contain the same *shape* of problems as real CHIETA data — missing identity numbers, duplicated learner records, inconsistent provider names, impossible enrolment dates — so that nothing pedagogical is lost.

Data ethics is taught as content, not as a disclaimer. The Lesson Plan's coverage of data lineage, privacy, ethical stewardship, bias, fairness and explainable AI is retained in full and distributed across Months 4, 5 and 6, where each topic can be attached to something the learner has actually built. A learner who has personally trained a dropout-risk model on synthetic learner data is in a position to have a real conversation about what it would mean to deploy such a model against real people; a learner who has only read about fairness is not.

---

3. The Case-Study Spine and the Competency Model

3.1 The spine: one artefact, twenty-four weeks

The organising device of this programme is a single continuously-built application, the **CHIETA Skills Data Lab (Mini)** — referred to throughout as *the Mini Lab*. It is a working, scaled-down implementation of the real CHIETA Data Lab, built by every learner, on synthetic data, against the same architecture the production system uses.

The Mini Lab is not a capstone that begins in Month 6. It begins in Week 1 and it never stops. Each month adds a layer, and each layer is unusable without the one beneath it — which is precisely what makes the sequence feel necessary to the learner rather than arbitrary.

Case study	Month	Title	What the learner builds	What it depends on	What later work depends on it
CS-0	Week 0	*The Journey*	A paper walkthrough of one CHIETA learner's path from enrolment to certification; a hand-drawn system diagram; a one-page personal learning plan	Nothing	Gives every later week a "who is this for" answer

CS-1	Month 1	*The Records*	A Python command-line tool that reads a messy learner CSV, validates and cleans it, computes completion rates by province, handles bad input without crashing, writes a clean CSV, and has passing pytest tests	CS-0	The cleaning logic becomes the ETL job in CS-4; the completion-rate function becomes an API endpoint in CS-3 and a model feature in CS-5
CS-2	Month 2	*The Shell*	A Next.js + TypeScript + Tailwind application: login screen, role-based navigation for Admin/Analyst/Viewer/Student, dataset list and detail pages, and a dashboard with three charts — all on mock data	CS-1 (the data shapes are the ones CS-1 produced)	Becomes the real UI once CS-3 supplies live APIs
CS-3	Month 3	*The Spine*	A FastAPI service with PostgreSQL, JWT login, RBAC-protected routes, dataset CRUD, audit logging, standard error envelopes, pagination, and Swagger docs — with CS-2's frontend switched from mock data to this live API	CS-1, CS-2	Everything after it. This is the load-bearing month
CS-4	Month 4	*The Pipeline*	An ETL job (extract → validate → clean → transform → load), MinIO object storage with raw/processed buckets, dataset versioning, an Airflow-style scheduled DAG, and a docker compose up that starts the whole stack	CS-1 (cleaning logic), CS-3 (the database it loads into)	Supplies the training data for CS-5 and the deployable images for CS-6
CS-5	Month 5	*The Insight*	Two full dashboards built on live SQL, a scikit-learn dropout-risk classifier with proper evaluation, MLflow experiment tracking, a POST /api/v1/predict endpoint, a prediction UI, and at least the backend deployed to Azure with secrets in Key Vault	CS-3, CS-4	Is the feature demonstrated in the CS-6 capstone
CS-6	Month 6	*The Product*	Automated tests (API + UI smoke), a CI/CD pipeline running on every pull request, container deployment, health monitoring, a PostgreSQL backup/restore drill, a UAT script with sign-off, full documentation, and the capstone demo	Everything	The handover artefact

Two structural notes on the spine. First, **each learner builds their own Mini Lab** — the artefact is individual, so assessment is individual — but from Month 4 each learner *also* owns a module in the shared CHIETA Data Lab codebase, following the Intern Training Plan's module-ownership model. The Mini Lab proves individual competence; the shared module proves the learner can work in a team codebase. Both are required. Second, **weekly mini-cases nest inside each monthly case study**; these are listed in the weekly breakdowns in Parts B–G and each one is a step in the current CS, never a detour.

3.2 The competency framework

The Lesson Plan left its "Education Standards" section as an unfilled placeholder. This is that section, filled. Competencies are grouped into six domains, each with observable levels, and every assessment in the programme maps to at least one. Full descriptors and the assessment-to-competency matrix are in Appendix E; the summary structure is:

Domain	Code	Covers	Assessed primarily in
Programming & Problem Solving	C1	Decomposition, control flow, data structures, functions, OOP, debugging, reading errors	Months 1–3
Data Engineering & Management	C2	SQL, schema design, ingestion, cleaning, validation, storage, lineage, versioning, orchestration	Months 1, 3, 4

Application Engineering	C3	Frontend, backend, APIs, auth/RBAC, integration, accessibility, error and loading states	Months 2, 3, 5
Analytics & AI	C4	EDA, statistics, model training and evaluation, experiment tracking, explainability, responsible AI	Month 5
Platform & Operations	C5	Containers, CI/CD, cloud deployment, secrets, monitoring, backup and restore, incident handling	Months 4, 6
Professional Practice	C6	Git and code review, agile ceremony, technical writing, help-seeking, peer review, stakeholder presentation, ethics and POPIA	All months, continuously

Each competency is rated on four levels — **Aware** (can explain it), **Assisted** (can do it with support), **Independent** (can do it alone), **Fluent** (can do it alone and teach it). Certification at Data Lab Associate requires *Independent* across C1–C5 and *Fluent* on the C6 sub-competencies for code review and documentation.

3.3 The badge ladder

The Beginner Course's internal badge ladder is retained, mapped to the competency framework, and tied to CHIETA Data Lab Project Handbook items FR-38/FR-39 (student contributions and digital badges):

Badge	Earned at	Minimum requirement
Data Lab Explorer	End of Month 2	CS-1 and CS-2 delivered; Month 1 and 2 skills checklists complete; C1 at Independent
API Builder	End of Month 3	CS-3 delivered with live frontend/backend integration; C3 at Assisted or better
Pipeline Packager	End of Month 4	CS-4 delivered; docker compose up runs the full stack; C2 at Independent
Cloud Analyst	End of Month 5	CS-5 delivered; a service reachable on a public HTTPS Azure endpoint; C4 at Assisted or better
Data Lab Associate	End of Month 6	Capstone demo passed, technical interview passed, all nine capstone requirements met, handover pack complete

---

4. Table of Contents

Page numbers are for the assembled 118-page handbook. Sections marked ★ are the two fully written sample pages provided in Section 16 of this blueprint.

FRONT MATTER — pages i–viii (numbered 1–8 in this outline)

Page	Content
1	Title page, version control block, document control table
2	Foreword from CHIETA and CXi Africa; programme sponsors and acknowledgements
3	How to use this handbook — three reading paths (learner / facilitator / programme manager)
4	Icon key, typographic conventions, code-block conventions, callout types
5	Design basis: what this programme is built on and why (condensed Section 2)
6	Source-document reconciliation table and the four resolved conflicts
7	Programme charter: purpose, scope, exclusions, success criteria

8	Quick-reference glossary of the 40 terms a Week-1 learner will meet

PART A — PROGRAMME ARCHITECTURE — pages 9–24

Page	Content
9	A.1 Programme purpose and the promise to the learner
10	A.2 Learner profile; what is and is not assumed
11	A.3 The Week 0 diagnostic and how results route learners to tracks
12–13	A.4 The competency framework (C1–C6) with four-level descriptors
14	A.5 The badge ladder and certification thresholds
15–16	A.6 The case-study spine — CS-0 to CS-6 explained in full
17	A.7 The 26-week map at a glance (single-page wall chart)
18	A.8 The two-track delivery model and hour budgets
19	A.9 The weekly rhythm — Monday to Friday, session by session
20	A.10 Instruction modes: classroom, lab, online, self-study, pair programming
21	A.11 Environment and toolchain: full install list, versions, sources
22	A.12 Hardware minimums, network requirements, low-resource fallbacks
23	A.13 Accounts, access, repository structure, branching model
24	A.14 Data safety, POPIA compliance, and the synthetic dataset policy

PART B — MONTH 1: FOUNDATIONS OF COMPUTING & PROGRAMMING — pages 25–38

Page	Content
25	B.0 Month 1 opener: theme, promise, and what the month feels like
26	B.1 Learning objectives and competency mapping
27	B.2 Week 0 — pre-programme onboarding: hardware, accounts, terminal literacy
28	B.3 Week 1 — computers, the web, and the Data Lab story
29	B.4 Week 2 — thinking like a developer; the command line; Git
30 ★	B.5 Week 3 — Python I: variables, types, control flow *(sample page provided)*
31	B.6 Week 3 continued — functions, scope, decomposition, DRY
32	B.7 Week 4 — Python II: lists, dictionaries, files, errors, modules, pytest
33	B.8 Week 5 — web basics and SQL basics; first PostgreSQL contact
34–35	B.9 Case Study CS-1 *The Records* — full brief, eleven build steps, acceptance criteria
36	B.10 Assessments, checkpoints and the Month 1 gate
37	B.11 Resources, readings and the Month 1 skills checklist
38	B.12 Facilitator notes, common misconceptions, and remediation routes

PART C — MONTH 2: FRONTEND ENGINEERING WITH NEXT.JS — pages 39–52

Page	Content
39	C.0 Month 2 opener and connection back to CS-1
40	C.1 Learning objectives and competency mapping
41	C.2 Week 6 — HTML semantics, CSS layout, accessibility foundations
42	C.3 Week 7 — JavaScript: arrays, objects, array methods, DOM, events, async

43	C.4 Week 8 — TypeScript: types, interfaces, unions, generics, tsconfig
44	C.5 Week 9 — React with TypeScript: components, typed props, lists, composition
45	C.6 Week 10 — Next.js App Router: layouts, routing, server vs client components
46	C.7 Week 10 continued — state, hooks, controlled inputs, custom hooks
47	C.8 Week 11 — auth UI, RBAC menus, the four roles, Zod + React Hook Form
48	C.9 Week 12 — Recharts, tables, loading/empty/error states, accessibility pass
49–50	C.10 Case Study CS-2 *The Shell* — full brief, twelve build steps, acceptance criteria
51	C.11 Assessments, checkpoints, Month 2 gate, Explorer badge
52	C.12 Resources, skills checklist, facilitator notes

CONSOLIDATION WEEK 1 — page 53

Page	Content
53	Mid-programme consolidation: catch-up protocol, extension menu, one-on-ones, re-diagnostic

PART D — MONTH 3: BACKEND ENGINEERING & DATA PERSISTENCE — pages 54–67

Page	Content
54	D.0 Month 3 opener: the return to Python as a confidence event
55	D.1 Learning objectives and competency mapping
56	D.2 Week 13 — Python intermediate: venvs, packages, OOP for services
57	D.3 Week 13 continued — HTTP, status codes, JSON, REST principles
58	D.4 Week 14 — FastAPI: routes, Pydantic models, dependency injection
59	D.5 Week 15 — PostgreSQL schema design: security, catalog, audit, ml
60	D.6 Week 15 continued — SQLAlchemy ORM, Alembic migrations, indexes
61	D.7 Week 16 — auth: password hashing, JWT issue/refresh, RBAC on the API
62	D.8 Week 16 continued — audit logging and the FR-34/FR-35 requirement
63	D.9 Week 17 — error envelopes, pagination, filtering, CORS, Swagger
64	D.10 Week 17 continued — connecting CS-2's frontend to the live API
65–66	D.11 Case Study CS-3 *The Spine* — full brief, fourteen build steps, acceptance criteria
67	D.12 Assessments, Month 3 gate, API Builder badge, resources, facilitator notes

PART E — MONTH 4: DATA ENGINEERING, STORAGE & CONTAINERS — pages 68–80

Page	Content
68	E.0 Month 4 opener and the shift from "learning" to "owning"
69	E.1 Learning objectives and competency mapping
70	E.2 Week 18 — dirty-data patterns, profiling, validation rules
71	E.3 Week 18 continued — lineage: raw → processed → warehouse → dashboard
72	E.4 Week 19 — MinIO, buckets, object storage, dataset versioning
73	E.5 Week 20 — orchestration: DAGs, operators, schedules, retries, idempotency
74	E.6 Week 20 continued — Pandas at scale; Polars and DuckDB as extension
75	E.7 Week 21 — Docker: images vs containers, Dockerfiles for Next.js and FastAPI
76	E.8 Week 21 continued — Compose, volumes, networks, env files, healthchecks

77	E.9 Data governance, stewardship and ethics applied to the Mini Lab
78–79	E.10 Case Study CS-4 *The Pipeline* — full brief, thirteen build steps, acceptance criteria
80	E.11 Assessments, Month 4 gate, Pipeline Packager badge, resources, facilitator notes

PART F — MONTH 5: ANALYTICS, MACHINE LEARNING & AZURE — pages 81–94

Page	Content
81	F.0 Month 5 opener: from storing data to answering questions with it
82	F.1 Learning objectives and competency mapping
83	F.2 Week 22 — exploratory data analysis and descriptive statistics
84	F.3 Week 22 continued — the five CHIETA dashboards; building two in full
85	F.4 Week 23 — ML fundamentals: features, labels, train/test split
86 ★	F.5 Week 23 continued — training and evaluating the dropout-risk model *(sample page provided)*
87	F.6 Week 24 — MLflow: experiment tracking, registry stages, artifacts
88	F.7 Week 24 continued — the /predict endpoint and the prediction UI
89	F.8 Responsible and explainable AI: bias, fairness, SHAP, communicating results
90	F.9 Week 25 — Azure fundamentals: subscriptions, resource groups, ACR
91	F.10 Week 25 continued — Container Apps deployment, Key Vault, cost awareness
92–93	F.11 Case Study CS-5 *The Insight* — full brief, fifteen build steps, acceptance criteria
94	F.12 Assessments, Month 5 gate, Cloud Analyst badge, resources, facilitator notes

PART G — MONTH 6: QUALITY, DEVSECOPS, CAPSTONE & HANDOVER — pages 95–108

Page	Content
95	G.0 Month 6 opener: month six is for finishing, not starting
96	G.1 Learning objectives and competency mapping
97	G.2 Week 26 — the testing pyramid; pytest for FastAPI
98	G.3 Week 26 continued — Playwright UI smoke tests; data-quality checks
99	G.4 Security: OWASP Top 10 awareness, dependency scanning, secure defaults
100	G.5 Week 27 — CI/CD: build → test → image → push → deploy; branching model
101	G.6 Week 28 — Kubernetes/AKS concepts, Helm, namespaces
102	G.7 Week 28 continued — observability, logging, P1–P4 incidents, backup/restore drill
103	Consolidation Week 2 — freeze, triage, rehearsal
104	G.8 Week 29 — UAT script, sign-off sheet, documentation pack
105	G.9 Week 30 — capstone demo day, technical interview, handover
106–107	G.10 Case Study CS-6 *The Product* — full brief, sixteen build steps, capstone acceptance criteria
108	G.11 Final assessment, Associate badge, reference letter, contribution report

PART H — THE ASSESSMENT SYSTEM — pages 109–115

Page	Content
109	H.1 Assessment philosophy and why there are no written exams
110	H.2 The weighting model and how marks are earned
111	H.3 Formative instruments: exit tickets, code review, stand-up, weekly demo

112	H.4 Summative instruments: monthly project demo, reflection, skills checklist
113	H.5 The four phase gates, minimum criteria, and the consequence of a missed gate
114	H.6 The capstone rubric and the technical-interview question bank
115	H.7 Moderation, appeals, reasonable accommodation, and record-keeping

PART I — FACILITATOR PLAYBOOK — pages 116–122

Page	Content
116	I.1 The facilitator's week; preparation checklist; session opening and closing scripts
117	I.2 Running a type-along lab with mixed-pace learners
118	I.3 The twenty misconceptions beginners have, and how to pre-empt each
119	I.4 Diagnosing a stuck learner: the four-question triage
120	I.5 Code review as teaching; how to write a review comment a beginner can act on
121	I.6 Running the monthly one-on-one; escalation and pastoral boundaries
122	I.7 Programme risks, early warning signs, and the intervention register

PART J — LEARNER SUPPORT, PROGRESSION & PROFESSIONAL PRACTICE — pages 123–128

Page	Content
123	J.1 The support ladder and how to ask a question that gets answered
124	J.2 The contribution log: what to record and why it becomes your reference letter
125	J.3 Study skills for beginners: spacing, retrieval, deliberate failure, note-taking
126	J.4 Working in a team codebase: branches, PRs, reviews, disagreement
127	J.5 Ethics, POPIA, and professional conduct with data
128	J.6 Career pathways after the programme; CV and portfolio construction

PART K — APPENDICES — pages 129–147

Page	Content
129–131	Appendix A — The synthetic dataset family and full data dictionary
132	Appendix B — Complete software install list with versions and sources
133	Appendix C — The 26-week master schedule (single-table reference)
134	Appendix D — Week 0 diagnostic instrument and scoring guide
135–137	Appendix E — Competency descriptors and the assessment-to-competency matrix
138–139	Appendix F — All rubrics: code quality, documentation, demo, peer review, interview
140	Appendix G — Templates: task breakdown, contribution log, help request, peer review
141	Appendix H — FR-1 to FR-40 coverage map against weeks
142	Appendix I — Project Handbook Volumes I–VIII mapping
143	Appendix J — Module briefs for the five learner roles
144	Appendix K — UAT script and sign-off sheet template
145	Appendix L — Handover checklist (Volume VII style)
146	Appendix M — Full glossary
147	Appendix N — Extension track menu and further reading

Total: 147 pages — comfortably exceeding the 80-page minimum, with the core instructional content (Parts A–G) occupying pages 9–108.

---

5. Detailed Outline — Front Matter (pages 1–8)

Purpose of this part. To orient three different readers in under ten minutes each, and to make the handbook's own logic legible so that instructors can edit it without breaking it.

Page	Subsections and page-level detail
1	Title block; document control table (version, date, author, approver, next review date); distribution list; a "this document supersedes" line naming the three source documents.
2	Foreword — half page from CHIETA on why skills development in the chemical sector needs data capability, half page from CXi Africa on the delivery commitment. Acknowledgement of the source authors, including the Intern Training Plan by Lethabo Molefe.
3	**How to use this handbook.** Three boxed reading paths. *Learner path:* pages 3, 7, 9–10, 17–19, then your current month's Part, then Part J. *Facilitator path:* pages 5–6, all of Part A, your month's Part in full, Part H, Part I. *Programme manager path:* pages 5–7, A.4–A.5, Part H, I.7, Appendices E, H, I, L. Explicit instruction: **never hand a beginner the CHIETA Data Lab Project Handbook in Week 1** — it is a map, not a reading assignment.
4	Conventions. Icon key: 🎯 objective, 🧪 lab, 💥 break-it-on-purpose step, ✅ checkpoint, 📎 deliverable, ⚠️ common error, 🌱 extension. Code-block conventions (language tag, filename header, $ for shell, >>> for REPL). Callout types: Concept, Why This Matters, Common Error, Facilitator Note, POPIA Note.
5	Condensed design basis — the five pedagogical principles and the two-track model, one paragraph each.
6	The source-reconciliation table and the four resolved conflicts, in the same form as Section 2.2 of this blueprint.
7	**Programme charter.** Purpose statement; scope (what is built); explicit exclusions (this is not a data-science degree; it does not certify against a specific NQF unit standard unless separately mapped; it does not cover Spark cluster administration, advanced statistics, or production SRE); success criteria (≥80% of enrolled learners reach Data Lab Associate; 100% of completers have a public portfolio artefact; ≥1 merged PR per learner in the shared codebase).
8	**Forty-term starter glossary** — the words a Week-1 learner will hear and not know: repository, commit, branch, pull request, terminal, path, environment variable, package, dependency, virtual environment, interpreter, runtime, framework, library, API, endpoint, request, response, status code, JSON, schema, table, row, column, primary key, foreign key, query, index, migration, container, image, registry, pipeline, deployment, environment (dev/staging/prod), secret, log, test, refactor, stack. One line each, no jargon inside the definitions.

---

6. Detailed Outline — Part A: Programme Architecture (pages 9–24)

Purpose of this part. To establish the contract between programme and learner before any teaching begins: what will be delivered, what is expected in return, how progress will be judged, and what the whole six months looks like on one page.

A.1–A.3 Purpose, learner profile, diagnostic (pages 9–11)

- **Page 9 — The promise.** A plain-language statement of what the learner will be able to do in six months, written in the second person, with the CS-6 capstone described concretely. Includes the honest counter-statement: what this programme will *not* make you (a senior engineer, a data scientist, a cloud architect) and what it *will* make you (a junior contributor who can be productive in a real data platform team from day one).
- **Page 10 — Learner profile.** Assumed: matric-level numeracy, functional English, confident general computer use, reliable access to a machine and connectivity, ~15 hrs/week. Not assumed: any programming, any command-line use, any mathematics beyond percentages and averages, any prior exposure to databases, cloud or version control. A short note on the two learner streams the programme serves — CHIETA-funded new entrants and TUT/WIL placement students — and why the material must work for both.
- **Page 11 — The Week 0 diagnostic.** Purpose (routing, not gatekeeping — nobody is excluded by it). Five short tasks: describe a process in ordered steps; find a file on your own machine and state its full path; read a small table and answer three questions about it; follow six written instructions exactly; explain in writing something you know well to someone who doesn't. Scoring bands and the resulting routing: Core Track, Core Track with Week-2 Python clinic, or Core + Extension. Instrument in Appendix D.

A.4–A.5 Competency framework and badges (pages 12–14)

- **Pages 12–13 — Competency framework.** Full descriptors for C1–C6 at all four levels (Aware / Assisted / Independent / Fluent), formatted as a matrix. Each descriptor is written as an observable behaviour, e.g. C1-Independent: *"Given a described problem, decomposes it into functions, writes working code without referring to prior solutions, and reads a stack trace to locate the failure."* This is the section that fills the Lesson Plan's empty "Education Standards" placeholder.
- **Page 14 — Badge ladder** and certification thresholds, cross-referenced to FR-38/FR-39.

A.6–A.7 The spine and the map (pages 15–17)

- **Pages 15–16 — The case-study spine.** CS-0 to CS-6 in full, each with: what is built, what it depends on, what depends on it, and — critically — the *one sentence a learner should be able to say* about that artefact at the end of the month. A diagram showing the Mini Lab growing layer by layer across the six months.
- **Page 17 — The 26-week map.** A single-page wall chart: weeks across, and rows for Month / Case Study / Primary language or tool / Weekly deliverable / Badge. Designed to be printed A3 and pinned to the lab wall. This is the single most-referenced page in the handbook.

A.8–A.10 Delivery model (pages 18–20)

- **Page 18 — Two-track hour budget.**

Activity	Core Track (hrs/wk)	Extension Track (additional)
Instructor-led theory (2 × 90 min)	3	—
Guided lab (2 × 2 hr)	4	+2
Build hour on the case study	4	+4

Self-study, video, reading	2	+2
Code review and peer feedback	1	—
Stand-up, wrap-up, contribution log	1	—
Total	**15**	**+8 → 23**

With a note: at 15 hrs × 26 weeks = 390 hours, which is sufficient for the full capstone at Core standard. Facilitators must plan lab content so the Core deliverable is completable inside the 4-hour lab plus the 4-hour build block; anything that requires more is an extension item and must be labelled 🌱.

- **Page 19 — The weekly rhythm**, adopted from the Intern Training Plan and extended:

Day	Activity	Lead	Duration
Monday	Stand-up: last week, this week, blockers. Each learner speaks.	Facilitator facilitates	30 min
Monday	Theory session 1 — concept demo for the week's first topic	Facilitator	90 min
Tuesday	Lab 1 — type-along, then independent exercise, then 💥 break-it step	Facilitator + assistant	2 hr
Wednesday	Deep work. No meetings. Case-study build block.	Self-directed	4 hr
Thursday	Theory session 2, then Lab 2	Facilitator	90 min + 2 hr
Thursday	Code review — PRs opened by Wednesday get reviewed	Facilitator + peers	1 hr
Friday	Weekly demo (5 min each), exit ticket, contribution log, wrap-up	Learner-led	1 hr

Session rules retained verbatim from the Intern Plan: 90-minute cap on taught sessions; every session ends with one exercise; the next session opens by reviewing it; two hours is the maximum time stuck before escalating; all code written in a session is committed and pushed the same day.

- **Page 20 — Instruction modes.** When to use each and why: **classroom/lab** for all new-concept introduction and all first contact with a tool (beginners cannot self-serve first contact); **online synchronous** acceptable for theory sessions from Month 3 onward once learners can debug independently; **self-study** for reference material, documentation reading, and extension work only — never for a new core concept; **pair programming** mandatory in Weeks 1–8 per the Beginner Course facilitator note, optional thereafter, with rotation rules and the driver/navigator protocol; **group work** for peer review (Week 19-equivalent), the shared-module work from Month 4, and UAT in Month 6.

A.11–A.14 Environment, hardware, access, data safety (pages 21–24)

- **Page 21 — Toolchain.** The install list, consolidated from the Intern Training Plan and extended for the fuller stack: VS Code (or Cursor), Python 3.12, Node.js 20 LTS, Git 2.40+, PostgreSQL 16, Docker Desktop, Azure CLI, Google Chrome. VS Code extensions: Python, Pylance, ESLint, Prettier, GitLens, Tailwind IntelliSense, Error Lens, Thunder Client, Live Server, Docker. Database and API tools: pgAdmin 4, DBeaver Community, Postman, React Developer Tools. Python packages: venv, pytest, black, requests, pandas, psycopg2-binary, sqlalchemy, alembic, fastapi, uvicorn, python-jose, passlib, scikit-learn, mlflow, minio. Node packages (per project, never global): typescript,

next, react, react-dom, tailwindcss, recharts, zod, react-hook-form, vitest, @testing-library/react, playwright. Every item has version, purpose, and source URL. **Nothing on the list costs money.**

- **Page 22 — Hardware and fallbacks.** Recommended 16 GB RAM / 50 GB free disk; workable at 8 GB with the documented light-lab path (SQLite instead of a Postgres container in Weeks 13–15; a scheduler stub instead of local Airflow in Week 20; cloud-hosted Postgres for Month 4 if Docker Desktop is unstable). Connectivity requirements and an offline-capable fallback bundle for labs where bandwidth is unreliable — a real constraint in South African delivery contexts and one that must be planned for, not improvised.
- **Page 23 — Accounts, repository structure, branching.** GitHub or Azure DevOps account; Azure free/student subscription; optional Docker Hub. The repository layout for the Mini Lab (/frontend, /backend, /data, /pipelines, /infra, /docs, /tests). Branching model — GitFlow-lite: main, develop, feature/* — with the naming convention and the PR template.
- **Page 24 — Data safety and POPIA.** The absolute rule: no real CHIETA personal information in any training environment, ever. What synthetic data is and why it is used. The synthetic dataset family (Appendix A). What a learner must do if they ever encounter real data by accident. A plain-language summary of the POPIA principles that apply to a data platform: purpose limitation, minimality, retention limits, security safeguards, data-subject rights. This page is referenced again in Months 4, 5 and 6.

---

7. Detailed Outline — Part B: Month 1, Foundations (pages 25–38)

Theme: From *"I have never coded"* to writing small programs, reading data, and understanding what the Data Lab is. **Project Handbook link:** Volume I (what we build), Volume VIII (how teams work). **Case study:** CS-1 *The Records*. **Badge progress:** Half of Data Lab Explorer.

B.0 Month opener (page 25)

A page written directly to the learner. It names the fear — *you are about to spend a month feeling stupid, and that is normal and temporary* — and reframes it: everyone in this room, including your facilitator, has stared at an error message and had no idea what it meant. It sets the month's single measure of success: **by Week 5 you will run a program you wrote yourself, on data you cleaned yourself, and it will tell you something true about CHIETA learners that you did not know before you started.**

B.1 Learning objectives (page 26)

By the end of Month 1, learners can:

1. Explain in plain language what the CHIETA Data Lab is, who uses it, and what the four user roles (Admin, Analyst, Viewer, Student) can each do. *(C6-Aware)*
2. Use a terminal to navigate a file system, create and move files, and run programs. *(C1-Independent)*

3. Use Git to clone, branch, commit, push, and open a pull request that is reviewed and merged. *(C6-Assisted)*
4. Write Python programs using variables, data types, conditions, loops, functions, lists and dictionaries. *(C1-Independent)*
5. Read a CSV file, handle malformed rows without crashing, and write results to a new file. *(C1-Independent, C2-Assisted)*
6. Write and run basic tests with pytest. *(C6-Assisted)*
7. Write SQL to select, filter, sort and join, and run those queries against PostgreSQL. *(C2-Assisted)*
8. Build a simple static HTML/CSS page and explain the client → server → database path. *(C3-Aware)*

B.2–B.8 Weekly breakdown (pages 27–33)

Week	Topics taught	Lab (🧪) and break-it step (💥)	📎 Weekly deliverable
0 *(onboarding)*	Hardware check, accounts, file systems, paths, keyboard and terminal literacy, what software is, how the internet moves a request. No programming.	🧪 Install the full toolchain with a facilitator present; screenshot each working tool. 💥 Deliberately mistype a path and read the "No such file or directory" error aloud.	Working toolchain screenshots + one-page personal learning plan (CS-0)
1	How software systems work: client → server → database. What the CHIETA Data Lab does (upload, clean, analyse, predict, report). The four user roles. What a program is; what an interpreter does. Variables, types, operators, input/output, f-strings, comments.	🧪 "Hello CHIETA" — a static HTML page and a first Python script, side by side, so learners see both halves of the stack on day one. 💥 Remove a closing quote; read the SyntaxError.	A running Python script that takes input and prints a formatted result; a rendered HTML page; both committed
2	Files, folders, paths, environment variables. Command-line essentials (cd, ls/dir, mkdir, cp, mv). Git: init, status, add, commit, branch, push, pull, PR, merge. Agile basics: backlog, sprint, user story in the form *"As an Analyst, I want…"*. Conditions, boolean logic, while and for loops, tracing code by hand.	🧪 Create chieta-learner-journal repo; commit daily notes; open a PR. 🧪 Trace three supplied programs **on paper** and predict output before running them. 💥 Create a merge conflict on purpose and resolve it.	Repo with ≥5 commits and one merged PR; three traced programs with predictions and actual output
3 ★	Defining functions, parameters, arguments, return values. Scope: local vs global and why globals cause pain. Default and keyword arguments. Docstrings. The DRY principle. Problem decomposition — planning on paper before coding.	🧪 Rewrite Week 2's grade calculator using functions. 🧪 Refactor a supplied script that repeats itself four times. 💥 Break a function by shadowing a variable; find it with a print-trace.	A decomposed, docstringed Python module of 5+ functions, with a written plan showing decomposition before code
4	Lists: index, slice, append, iterate, methods.	🧪 Read a deliberately	clean_learners.py v1

	Dictionaries: key-value, updating, looping. Tuples and sets; deduplication. Nested structures — lists of dictionaries, *which is how real data arrives*. try/except/finally; common exception types; failing safely. File I/O with with; the csv module; a first look at pandas. Modules and imports. What a test is; first pytest test.	messy learner CSV; count records; find the duplicates. 💥 Feed the script a row with a missing column and watch it crash; then make it survive.	that reads the messy CSV, survives bad rows, and prints a summary; 3 passing pytest tests
5	HTML5 structure, semantic tags, tables, forms. CSS selectors, the box model, basic layout. JavaScript orientation only: variables, a function, a DOM update. Relational concepts: tables, rows, columns, primary and foreign keys, why data is split. SQL: SELECT, WHERE, ORDER BY, LIMIT, INSERT, UPDATE, INNER/LEFT JOIN, GROUP BY, COUNT/SUM/AVG. PostgreSQL locally or via Docker; pgAdmin.	🧪 Build a static "Learner KPI card" page. 🧪 Load the cleaned CSV into a Postgres table and write eight queries, ending with completion rate by province. 💥 Write a query with a wrong join key and explain why the row count exploded.	A static KPI page; a queries.sql file with 8 working queries and commented explanations

B.9 Case Study CS-1 — *The Records* (pages 34–35)

Scenario given to the learner. CHIETA's regional offices submit learner enrolment and completion records as spreadsheets. The files arrive inconsistently: some have missing provinces, some repeat the same learner twice, some record completion dates that fall before the enrolment date, and provider names are spelled four different ways. Before anyone can answer the question *"which provinces are we underserving?"*, someone has to clean this. That is you.

What you build. A Python command-line tool, clean_learners.py, plus a small SQL script and a short written summary.

Explicit build steps:

1. **Read the brief and write the user story.** *"As a CHIETA Analyst, I want clean learner records so that I can compare completion rates by province."* Written before any code. *(Week 2)*
2. **Plan on paper.** List the transformations needed, in order, as function names. No code yet. *(Week 3)*
3. **Load the file.** Open learners_raw.csv with the csv module; print the first five rows; print the column names. *(Week 4)*
4. **Validate rows.** Write is_valid_row(row) returning True/False plus a reason. Rules: learner ID present and non-blank; province in the known list of nine; enrolment date parseable; completion date, if present, not earlier than enrolment date. *(Week 4)*
5. **Handle the failures.** Route invalid rows to learners_rejected.csv with the rejection reason appended, rather than dropping them silently. **This is the single most important habit in the whole month** — data that fails validation is evidence, not garbage. *(Week 4)*
6. **Deduplicate.** Write deduplicate(rows) using a set of learner IDs, keeping the most recently updated record. *(Week 4)*
7. **Normalise.** Write normalise_provider(name) to collapse spelling variants using a mapping dictionary. Strip whitespace; title-case province names. *(Week 4)*

8. **Compute.** Write completion_rate_by_province(rows) returning a dictionary of province → percentage. This exact function is the ancestor of an API endpoint in CS-3 and a model feature in CS-5. *(Week 4–5)*
9. **Guard the whole thing.** Wrap file access in try/except; give a clear message if the file is missing rather than a stack trace. *(Week 4)*
10. **Write it out.** Produce learners_clean.csv, then load it into PostgreSQL and reproduce the completion-rate calculation in SQL. Compare the two answers; they must match. *(Week 5)*
11. **Test it.** Three pytest tests minimum: one for is_valid_row with a known-bad row, one for deduplicate, one for completion_rate_by_province against a hand-calculated expected value. *(Week 5)*

Acceptance criteria (all must pass): runs from the terminal with a filename argument; never crashes on the supplied dirty file; produces both clean and rejected outputs; Python and SQL completion rates agree to one decimal place; three tests pass; every function has a docstring; committed on a feature branch and merged via a reviewed PR.

Forward links. Step 4's validation rules become the Great-Expectations-style checks in CS-4. Step 7's normalisation becomes a transform stage in the ETL DAG. Step 8's function is exposed as GET /api/v1/analytics/completion-by-province in CS-3.

B.10 Assessments and the Month 1 gate (page 36)

Formative (weekly): Friday exit ticket (three questions: one recall, one apply, one "what confused you"); Friday 5-minute demo; code review on every PR; the stand-up self-report.

Summative (month-end): CS-1 demonstrated live and graded against the acceptance criteria; a 400-word written reflection; the Month 1 skills checklist signed off.

Gate criteria — must be met to proceed to Month 2: - Toolchain working; can run the project locally - At least one PR opened, reviewed and merged - Can write, unaided, a ~50-line Python program that takes input, processes it with functions and loops, and prints a result - CS-1 runs without crashing on the supplied dirty data

If the gate is missed: the learner enters the **Python clinic** — two extra 90-minute sessions per week during Month 2, targeting the specific weak area only, with a re-gate at the end of Week 8. Month 2 is *not* delayed for that learner; the frontend work in Month 2 does not depend on deep Python fluency, so proceeding while remediating is safe. This is a deliberate departure from the Intern Training Plan's "delay month 2 by one week", which stalls the whole cohort.

B.11 Resources and skills checklist (page 37)

Core (supplied): this handbook; the synthetic dataset family (Appendix A); the annotated clean_learners_reference.py; the Week 0 install guide; the Git cheat-sheet card. **Reference (free, external):** the official Python tutorial (docs.python.org); the Git Book chapters 1–3; MDN's HTML and CSS getting-started guides; PostgreSQL tutorial chapters 1–2; pytest getting-started. Facilitators should link these, not print them. **Project context:** Project Handbook Volume I (SRS) — sections on user roles and functional modules only. **Not** the full handbook.

Month 1 skills checklist: ☐ Git commit and merged PR ☐ Python script that cleans a CSV ☐ Basic SQL queries including one join ☐ Can explain Admin / Analyst / Viewer / Student ☐ Can name the main stack layers ☐ Three passing pytest tests ☐ Can read a stack trace and locate the failing line

B.12 Facilitator notes (page 38)

- **The five hardest moments in Month 1**, in order: the first terminal session (Week 0), the first merge conflict (Week 2), the difference between = and == (Week 2), scope confusion with globals (Week 3), and the first nested list-of-dictionaries (Week 4). Budget extra time at each.
- **Common misconceptions:** that a variable holds a *name* rather than a value; that indentation is decoration; that a function must print rather than return; that a for loop over a dictionary yields values (it yields keys); that a crash means the learner is bad at this.
- **Pair programming is mandatory this month.** Rotate pairs weekly. Enforce the driver/navigator swap every 15 minutes with a visible timer.
- **Never fix a beginner's code by taking the keyboard.** Ask: what did you expect? what happened? what's the first line of the error? where is that line in your file? This is the four-question triage from I.4 and it should be modelled from Week 1 so learners internalise it.
- **Motivation watch.** Weeks 3 and 4 are the highest-risk weeks for dropout in the entire programme — the novelty of Week 1 has worn off and nothing visual has been built since. Mitigate by front-loading the Week 5 KPI page as a preview in Week 3: show them what they will build in two weeks.

Expected outcomes at the end of Month 1

A learner who completes Month 1 can sit at an unfamiliar machine, set up a project, write a Python program that processes a real (if synthetic) dataset without crashing, prove it works with tests, get it reviewed and merged by another person, and query the same data in SQL to check their own answer. They have shipped something visible five weeks in a row. Critically, they have a working relationship with error messages: they read them instead of fearing them.

---

8. Detailed Outline — Part C: Month 2, Frontend Engineering (pages 39–52)

> **Theme:** Build the screens users see. **Project Handbook link:** Volume III (React frontend design); Modules 1–3 and 6–7 UI surfaces; FR-1 to FR-8, FR-16 to FR-19. **Case study:** CS-2 *The Shell*. **Badge:** Data Lab Explorer at month end.

C.0 Month opener and the link back to CS-1 (page 39)

Month 1 ended with a table of completion rates printed to a terminal. Nobody outside this room will ever look at a terminal. Month 2 exists to answer the question the learner is now ready to ask: *how does that number get in front of an Analyst who has never heard of Python?* The month opens by taking the exact output of CS-1 — the same provinces, the same percentages — and

rendering it as a KPI card in a browser. The continuity is the point: this is not a new project, it is a new *surface* on the same project.

C.1 Learning objectives (page 40)

By the end of Month 2, learners can: build multi-page applications with the Next.js App Router *(C3-Independent)*; write typed React components with TypeScript interfaces for User, Dataset and KPI *(C3-Independent)*; style with Tailwind against a defined token set *(C3-Assisted)*; implement a login UI and role-gated navigation for the four roles *(C3-Assisted)*; render dataset list and detail pages and a dashboard with at least three charts *(C3-Independent)*; handle loading, empty and error states as first-class UI *(C3-Independent)*; apply accessibility basics — labels, contrast, keyboard order *(C3-Assisted, C6-Aware)*; and explain why a large system uses static typing *(C1-Independent)*.

C.2–C.9 Weekly breakdown (pages 41–48)

Week	Topics taught	💉 Lab and 💥 break-it step	🪦 Deliverable
6	Client and server revisited in the browser: request, response, status codes, headers in DevTools. HTML5 semantics, headings, lists, tables, forms, attributes. CSS selectors, specificity, the cascade. The box model. Flexbox and Grid. Responsive design with media queries. Accessibility: labels, alt text, contrast, focus order.	💉 Inspect a real request in Chrome DevTools and annotate each part. 💉 Build the static markup and styling for a dashboard page. 💥 Fix a supplied layout broken by box-sizing.	A responsive static dashboard page working at three screen widths
7	JavaScript proper: let/const, types compared to Python, function declarations vs arrow functions, callbacks. Arrays and objects; destructuring. map, filter, reduce, find, sort and chaining. The DOM: select, change, create, remove. Events and handlers. Promises, async/await, fetch.	💉 Translate three Month-1 Python scripts into JavaScript — the direct bridge from what they know. 💉 Populate an HTML table from an array of learner objects; add a working filter dropdown. 💥 Forget await and observe a Promise printed instead of data.	An interactive HTML table with a working filter, driven by CS-1's cleaned data
8	Why TypeScript: the problems it solves on a team. Basic types; any and why to avoid it; inference. Interfaces vs type aliases. Union and literal types. Optional and readonly properties. Typed function parameters and returns; void. Generics as reusable typed containers. tsconfig.json and strict mode. Compilation.	💉 Break a JavaScript file deliberately, then watch TypeScript catch the same error before running. 💉 Add types to every Week-7 exercise. 💉 Scaffold a Vite/Next TypeScript project from scratch. 💥 Assign a string to a number and read the compiler message.	A typed project scaffold; Learner, Provider and KPI interfaces defined
9	React components and TSX. Typed props and props interfaces; default values. Rendering lists with map; why keys matter. Conditional rendering; the empty state. Composition, children props, keeping components small.	💉 Build a KPICard component from scratch taking label, value, trend as typed props. 💉 Assemble a dashboard row from three KPICards. 💥 Remove the key prop from a list and observe the console warning and the reordering bug.	A reusable, typed KPICard rendering CS-1's real completion figures
10	Next.js App Router: the app/ directory, layouts,	💉 Scaffold /login and	A routed app

	pages, nested routes, Link. Client vs server components — practical rules for beginners. Environment variables (NEXT_PUBLIC_*). Then state: useState and typing state; useEffect, dependency arrays and cleanup; controlled inputs and typed event objects; lifting state to a shared parent; extracting a custom hook.	/dashboard routes with a shared layout. 💉 Build a filter bar (text input + two dropdowns) that updates a table beside it. 💉 Write useFetch and use it in two components. 💥 Omit a useEffect dependency and watch the stale value.	shell; a working filter bar driving a table; one custom hook
11	Authentication UI: the login form, mock JWT, where a token lives and the trade-offs. Role-based menus for Admin / Analyst / Viewer / Student. The Data Lab page set: Datasets, Explore, Reports, Settings. Form validation with Zod and React Hook Form.	💉 Wire four demo accounts and prove each role sees a different menu. 💉 Add Zod validation with inline error messages. 💥 Log in as Viewer and attempt to reach an Admin route; make the app refuse it gracefully.	Role-gated navigation working for all four roles, with validated login
12	Recharts: bar, line and pie for Learner and Grant-style KPIs; passing typed data; axis labelling. Tables: sorting, pagination, density. Loading skeletons, empty states, error states — designed, not accidental. Accessibility pass: labels, contrast ratios, keyboard navigation, chart alternatives for screen readers.	💉 Build the completion-rate chart from CS-1's data. 💉 Add loading, empty and error states to every data surface. 💉 Run a keyboard-only navigation audit. 💥 Return an empty array from the mock API and confirm the empty state renders rather than a blank page.	Dashboard with ≥3 charts, all three data states handled, keyboard-navigable

C.10 Case Study CS-2 — *The Shell* (pages 49–50)

Scenario. CHIETA has agreed the Data Lab's four user roles. Before any backend exists, the team needs a clickable frontend that proves the navigation, permissions and dashboard layout are right — so that stakeholders can react to something real instead of a wireframe. Mock data first, real integrations second.

Explicit build steps:

1. Scaffold a Next.js 14 App Router project with TypeScript and Tailwind. Commit the scaffold before writing a single feature. *(Week 8)*
2. Define the type contracts first: User, Role, Dataset, KPI, CompletionByProvince. **These interfaces must match the shape CS-1 produced** — this is the moment the learner sees a data contract span two languages. *(Week 8)*
3. Create mockData.ts populated from the real output of CS-1. Not lorem ipsum; the learner's own cleaned figures. *(Week 9)*
4. Build the design-system primitives: KPICard, Badge, Button, EmptyState, Skeleton. *(Week 9)*
5. Build the app shell: root layout, header, sidebar, and the four routes /dashboard, /datasets, /explore, /settings. *(Week 10)*
6. Build /login with a mock JWT, storing the role and rendering menus conditionally. *(Week 11)*
7. Implement role gating: define a permission map, hide unavailable navigation, and refuse direct URL access with a clear message rather than a crash. *(Week 11)*

8. Build /datasets as a searchable, filterable list with a detail drawer. *(Week 11)*
9. Build the dashboard: three KPICards plus a bar chart (completion by province), a line chart (enrolments over time) and a table (recent uploads). *(Week 12)*
10. Add loading, empty and error states everywhere data appears. Prove each by toggling a flag in the mock layer. *(Week 12)*
11. Accessibility pass: every input labelled, contrast checked, full keyboard traversal, chart data also available as a table. *(Week 12)*
12. Write the README: what it is, how to run it (npm install, npm run dev), what is mocked, what comes next. *(Week 12)*

Acceptance criteria: runs from a clean clone with the documented commands; all four roles demonstrably see different navigation; ≥3 charts render from typed mock data; no any in the codebase; all three data states demonstrable; keyboard-navigable; README complete; merged via reviewed PR.

Forward links. Step 2's interfaces become the Pydantic response models in CS-3 — learners will discover their frontend types and backend schemas must agree, which is the most valuable lesson of Month 3. Step 3's mock layer is swapped for live fetch calls in Week 17 and the mock file is *kept*, because it becomes the test fixture in CS-6.

C.11 Assessment, gate and badge (page 51)

Formative: Friday demo of the week's UI; PR review focusing on component size and prop typing; a mid-month peer walkthrough where each learner explains one component to another.

Summative: CS-2 demo against acceptance criteria; the Month 2 skills checklist; a 400-word reflection on the difference between JavaScript and TypeScript *as experienced*, not as defined.

Month 2 gate: can create a new Next.js page and component unaided; has written TypeScript interfaces for User, Dataset and KPI; role-gated routes work; dashboard has ≥3 charts; README runs. **Missed gate** → targeted React clinic during Month 3 (React does not block Month 3's backend work, so the learner proceeds).

Badge awarded: Data Lab Explorer — Months 1 and 2 complete.

C.12 Resources and facilitator notes (page 52)

Resources: Next.js App Router docs (getting started and routing only); React docs "Learn React" first six chapters; TypeScript Handbook "Everyday Types" and "Object Types"; Tailwind docs core concepts; Recharts examples gallery; WebAIM contrast checker; Project Handbook Volume III, frontend sections.

Facilitator notes. Two failure modes dominate this month. First, **learners copy component code without reading it** — counter this by requiring, at every Friday demo, that the learner delete one line of their own component live and predict what breaks. Second, **TypeScript is experienced as an obstacle rather than a tool** — counter this by never introducing a type before the learner has hit the bug it prevents; Week 7's untyped JavaScript exists precisely to manufacture that bug. Watch for the learner who spends four hours on CSS alignment; cap

styling work explicitly and move them on. Pair programming remains mandatory through Week 8 and becomes optional from Week 9.

Expected outcomes at the end of Month 2

The learner has a running application they can open in a browser and hand to a stakeholder. They can explain the difference between a component and a page, why a type contract matters when two people work on the same system, and why an empty state is a design decision rather than an oversight. They have shipped visible progress eleven weeks in a row.

---

9. Consolidation Week 1 (page 53)

Placed after Week 12, at the exact midpoint. It is not a holiday and it is not new content.

Structure of the week:

- **Monday:** cohort re-diagnostic against the C1–C3 competency descriptors; each learner self-rates, the facilitator moderates, disagreements are discussed individually.
- **Tuesday–Wednesday:** two parallel tracks. **Catch-up track** — learners below gate criteria work only on the specific gap, with the facilitator, on the specific artefact that is incomplete. **Extension track** — learners at or above gate choose from the 🌱 menu: Apache Superset connected to the CS-1 dataset, Polars and DuckDB as pandas alternatives, advanced Tailwind theming, or beginning the Month 3 FastAPI material early.
- **Thursday:** monthly one-on-ones for the whole cohort (30 minutes each, per the Intern Training Plan) — not progress reviews, but honest conversations about workload, support level, and where the learner wants this to take them.
- **Friday:** cohort retrospective — what has worked, what has not, what changes in the second half. The facilitator commits publicly to at least two changes arising from it.

Why this week exists. Without it, the learners who fell behind in Month 1 arrive at Month 3 — the load-bearing month — carrying a deficit they cannot recover from while new material lands weekly. Every programme that skips this week loses its bottom quartile in Month 4.

---

10. Detailed Outline — Part D: Month 3, Backend & Data Persistence (pages 54–67)

Theme: Build the APIs and the business logic. **Project Handbook link:** Volumes III–IV (FastAPI services, API catalogue, PostgreSQL); FR-1 to FR-8, FR-34 to FR-37. **Case study:** CS-3 *The Spine*. **Badge:** API Builder. ⚠️ **This is the load-bearing month.** Everything in Months 4–6 assumes it.

D.0 Month opener (page 54)

Written to exploit a deliberate structural payoff: the learner spent Month 1 learning Python and then two months apparently leaving it behind. Month 3 opens by revealing that the CHIETA Data Lab's entire backend is Python — the language they learned first is the language the system actually runs on. Framed explicitly as a confidence event: *you already know the language this month is written in.*

D.1 Learning objectives (page 55)

By the end of Month 3, learners can: build REST APIs with FastAPI and Pydantic *(C3-Independent)*; design a normalised PostgreSQL schema with appropriate keys and indexes *(C2-Independent)*; connect Python to Postgres through SQLAlchemy and manage change with Alembic migrations *(C2-Assisted)*; implement password hashing, JWT issue and refresh, and RBAC on API routes *(C3-Assisted)*; return a standard error envelope and paginate and filter list endpoints *(C3-Independent)*; document an API with Swagger/OpenAPI *(C6-Independent)*; write an audit log entry for security-relevant actions *(C2-Assisted, C6-Aware)*; and connect the Month 2 frontend to this live backend, resolving the CORS and contract mismatches that arise *(C3-Independent)*.

D.2–D.10 Weekly breakdown (pages 56–64)

Week	Topics taught	🧪 Lab and 💥 break-it step	🖱️ Deliverable
13	Python intermediate: virtual environments, requirements.txt, pinned versions, uvicorn. OOP for services: classes, __init__ and self, methods, inheritance, when *not* to inherit. Modelling the domain — Learner, Provider, Grant — as classes. Then HTTP properly: methods, status codes, headers, JSON request and response bodies, idempotency, REST resource naming.	🧪 Model Learner and Provider classes with methods that compute a completion rate. 🧪 Use Postman to hit a public API and annotate every part of the request and response. 💥 Send a POST to a GET-only route and read the 405.	Domain classes with tests; an annotated Postman collection
14	FastAPI: application object, routers, path and query parameters, request and response models with Pydantic, validation errors, dependency injection, project structure (routers/, models/, schemas/, services/, db/). Auto-generated Swagger.	🧪 GET /health; GET /api/v1/datasets from an in-memory list. 🧪 Add a Pydantic response model and see Swagger update itself. 💥 Return a field the response model doesn't declare and observe the validation error.	A running FastAPI service with two documented endpoints and live Swagger
15	PostgreSQL schema design for the Data Lab: the security, catalog, audit and ml schemas (simplified). Tables: users, roles, datasets, dataset_versions, audit_logs. Keys, constraints, normalisation, snake_case naming, indexes and when they help. Then SQLAlchemy: models, sessions, relationships, queries. Alembic: what a migration is, autogenerate, upgrade, downgrade.	🧪 Draw the ER diagram on paper before writing DDL. 🧪 Create the schema, seed an Admin and an Analyst, write the first migration. 💥 Add a NOT NULL column to a populated table without a default; read the error; fix it with a two-step migration.	Migrated schema in Postgres; seeded users; ER diagram committed to /docs
16	Password hashing with bcrypt/passlib — and why plain text or MD5 is a career-ending mistake. JWT: structure, claims,	🧪 Implement POST /api/v1/login returning a JWT; protect a route by role. 🧪 Write the audit-log insert and	JWT auth working; three role-protected

	expiry, signing, refresh. Login and refresh endpoints. RBAC as a FastAPI dependency. CRUD /api/v1/users; upload/list/get for /api/v1/datasets, saving files to local disk for now. Audit logging on login and upload (FR-34/FR-35).	query it back. 💥 Tamper with a JWT payload and confirm signature verification rejects it.	endpoints; audit rows visible
17	The standard error envelope (timestamp, status, error, message, path) and a global exception handler. Pagination (page, size) and filtering (category, tag, date range). CORS and why the browser blocks the first request. Then integration: replacing CS-2's mock layer with live fetch calls, typing the responses, handling 401 and refresh.	💉 Add the exception handler; force a 404, a 422 and a 500 and confirm the envelope shape. 💉 Swap the frontend from mock to live and fix every mismatch. 💥 Deliberately mismatch a field name between the Pydantic model and the TypeScript interface; experience the runtime failure that static typing on one side alone does not prevent.	Frontend running entirely on live API; consistent error envelopes; contract mismatches documented

D.11 Case Study CS-3 — *The Spine* (pages 65–66)

Scenario. The clickable shell convinced stakeholders. Now it needs to be real: users who actually exist, permissions that are actually enforced, datasets that actually persist, and an audit trail that proves who did what.

Explicit build steps:

1. Create the backend project structure and a virtual environment; pin dependencies in requirements.txt. *(Week 13)*
2. Design the schema on paper as an ER diagram; have it reviewed *before* writing DDL. *(Week 15)*
3. Implement SQLAlchemy models for users, roles, datasets, dataset_versions, audit_logs. *(Week 15)*
4. Create the initial Alembic migration; run it; verify in pgAdmin. *(Week 15)*
5. Seed one user per role: Admin, Analyst, Viewer, Student. *(Week 15)*
6. Implement POST /api/v1/login with bcrypt verification returning a signed JWT plus refresh token. *(Week 16)*
7. Implement the RBAC dependency and apply it to routes; write the permission matrix into /docs. *(Week 16)*
8. Implement CRUD /api/v1/users, restricted to Admin. *(Week 16)*
9. Implement POST /api/v1/datasets (multipart upload to local disk plus a metadata row), GET /api/v1/datasets (paginated, filterable) and GET /api/v1/datasets/{id}. *(Week 16)*
10. Expose GET /api/v1/analytics/completion-by-province — **the CS-1 function, now an endpoint.** Point this out explicitly in class. *(Week 16)*
11. Write an audit-log row on every login and every upload, capturing actor, action, target, timestamp and IP. *(Week 16)*
12. Add the global exception handler and the standard error envelope. *(Week 17)*
13. Configure CORS; switch the CS-2 frontend from mockData.ts to live fetch; fix every contract mismatch and record each one in /docs/contract-issues.md. *(Week 17)*

14. Verify Swagger documents every endpoint accurately; export the OpenAPI spec into /docs. *(Week 17)*

Acceptance criteria: uvicorn starts cleanly; Swagger lists every endpoint with correct models; all four seeded roles behave per the permission matrix; a Viewer receives 403, not 500, on a forbidden route; upload persists file and metadata; the frontend runs with zero mock data; audit rows exist for every login and upload; migrations run from empty on a fresh database; error envelope identical across 404, 422 and 500.

Forward links. The upload handler's local-disk write is replaced by MinIO in CS-4 — a deliberate two-step so learners understand what object storage replaces. The audit table becomes the monitoring source in CS-6. The completion-by-province endpoint becomes a model feature source in CS-5.

D.12 Assessment, gate, badge, resources, facilitator notes (page 67)

Gate criteria (Month 3): can explain REST and JSON; can create and query PostgreSQL tables; login returns a working JWT; dataset metadata uploads via the API; the same data appears in Next.js from the live API. Per the Intern Training Plan, at least **two tasks merged into the shared project codebase** by this point.

Missed gate → this is the one gate that cannot be deferred, because Month 4 builds directly on a working API and database. A learner below gate is paired with the facilitator for the whole of Week 18 and given a reduced-scope CS-4 (Compose with three services instead of four, no dataset versioning). They are not moved back a month; they are given a narrower path forward.

Badge: API Builder.

Resources: FastAPI tutorial (first-steps through security); Pydantic v2 concepts; SQLAlchemy 2.0 ORM quickstart; Alembic tutorial; PostgreSQL docs on constraints and indexes; OWASP cheat sheets on password storage and JWT; Project Handbook Volume IV (DAES) and the API catalogue in Volume III.

Facilitator notes. The two hardest ideas this month are **dependency injection** (teach it as "FastAPI hands you the thing you asked for" before ever using the term) and **the async/sync boundary** (for beginners: use def unless you have a reason, and state the reason). Expect the CORS error in Week 17 to consume ninety minutes — plan for it; it is one of the most valuable teachable moments in the programme because it forces the learner to reason about two running processes and a browser policy simultaneously. Do not let learners store JWTs in localStorage without discussing the trade-off explicitly; this is the first genuine security decision they make.

Expected outcomes at the end of Month 3

The learner has a full-stack application: a browser UI they built, talking to an API they built, reading from a database they designed, with authentication and permissions they implemented, and an audit trail proving it. They can trace a single click in the browser all the way to a SQL query and back, naming every layer it passes through. This is the moment the programme's promise stops being abstract.

11. Detailed Outline — Part E: Month 4, Data Engineering, Storage & Containers (pages 68–80)

Theme: Move, clean, store and package the platform. **Project Handbook link:** Volumes II–V (ETL, MinIO, Docker Compose); FR-9 to FR-15, FR-28 to FR-31. **Case study:** CS-4 *The Pipeline*. **Badge:** Pipeline Packager.

E.0 Month opener (page 68)

Month 4 is where the programme changes character. Until now, learners have been taught and have then applied. From Week 18 the Intern Training Plan's **module ownership** model activates: each learner is additionally assigned a module in the *shared* CHIETA Data Lab codebase, with a written brief, and is expected to break it into tasks, plan the order, raise blockers early, and drive the work themselves. Taught sessions continue, but the balance shifts from instruction to supported autonomy. The opener explains this shift plainly so learners are not surprised by the reduced hand-holding, and sets the expectation that flagging a problem with a brief is a *sign of competence*, not of difficulty.

Role assignment (frontend, UI/UX, data/backend, full-stack, QA/documentation) happens at the start of Week 18, based on demonstrated strengths in Months 1–3. Module briefs are in Appendix J. Every learner, regardless of role, must contribute at least one PR to the frontend and must be able to explain how data moves through the system.

E.1 Learning objectives (page 69)

By the end of Month 4, learners can: identify and remediate dirty-data patterns systematically *(C2-Independent)*; write declarative validation rules and route failures to a quarantine path *(C2-Independent)*; explain and demonstrate lineage from raw file to dashboard *(C2-Independent)*; store and retrieve objects in MinIO with a raw/processed bucket strategy *(C2-Assisted)*; implement dataset versioning *(C2-Assisted)*; design a DAG with operators, schedules, retries and idempotency *(C2-Assisted)*; write Dockerfiles for a Next.js and a FastAPI service *(C5-Independent)*; orchestrate the full stack with Docker Compose including volumes, networks, env files and healthchecks *(C5-Independent)*; and articulate the governance and ethical obligations attaching to a dataset *(C6-Independent)*.

E.2–E.9 Weekly breakdown (pages 70–77)

Week	Topics taught	🧪 Lab and 💥 break-it step	🕯️ Deliverable
18	Dirty-data patterns: nulls, duplicates, wrong types, outliers, inconsistent categoricals, impossible dates, encoding problems, trailing whitespace. Data profiling: shape, cardinality, null rate, distribution. Validation as declarative rules (Great Expectations concepts). Quarantine over deletion. Then lineage: raw → processed → warehouse → dashboard, and why every number on a dashboard must be traceable to a file.	🧪 Profile the messy CHIETA learners file and produce a written data-quality report. 🧪 Convert CS-1's imperative validation into a declarative rule set. 💥 Introduce a silent type coercion (province stored as a number) and trace how far it travels before anyone notices.	A data-quality report; a rule-set file; a hand-drawn lineage diagram for one KPI
19	Object storage: why files do not belong in a relational database. MinIO as S3-compatible storage. Buckets: raw-data, processed-data, reports, models. Keys, prefixes, presigned	🧪 Replace CS-3's local-disk write with a MinIO put; store metadata in Postgres; preview the file in the	End-to-end upload → MinIO → DB row → frontend preview,

	URLs. Dataset versioning: immutable objects, version rows in Postgres, "latest" as a pointer.	frontend. 🧪 Upload the same dataset twice and prove v1 is still retrievable. 💥 Delete the object but not the metadata row; observe the broken link; then implement the integrity check.	with versioning
20	Orchestration: what a scheduler does; a DAG as a graph of tasks; operators; schedule intervals; retries and backoff; idempotency and why re-running must be safe. Local Airflow, or the lightweight scheduler stub on constrained hardware. Then Pandas at scale: read_csv options, dtypes, chunking, memory. 🌱 Extension: Polars and DuckDB as faster alternatives; Spark *concepts* only — when data outgrows one machine.	🧪 Build a DAG: extract from MinIO → validate → clean → transform → load to Postgres, scheduled nightly. 🧪 Run it twice and prove the result is identical. 💥 Make a task fail mid-DAG; observe the retry; then make the DAG resume correctly rather than duplicating rows.	A working scheduled pipeline with retries, proven idempotent
21	Containers: images vs containers; the Dockerfile as a build recipe; layers and caching; base image choice; .dockerignore. A Dockerfile for FastAPI and one for Next.js, in the Project Handbook's style. Then Compose: services, depends_on, volumes, networks, .env files, healthchecks, port mapping.	🧪 Containerise the backend alone and run it. 🧪 Write docker-compose.yml for frontend + backend + postgres + minio; bring the whole stack up with one command. 💥 Remove a service's network alias and watch the connection refuse; read the error and fix it.	docker compose up starts the entire Mini Lab from a clean clone
—	**(Page 77, taught across the month)** Data governance and ethics applied: who owns a dataset, retention, purpose limitation, minimality, the difference between de-identified and anonymous, POPIA obligations for a data platform, and what a data steward actually does day to day. Delivered as one 90-minute session in Week 19 and revisited in every code review this month.	🧪 Write a one-page data-stewardship statement for the Mini Lab's learner dataset: what it contains, who may see it, how long it is kept, how it is destroyed.	A stewardship statement committed to /docs

E.10 Case Study CS-4 — *The Pipeline* (pages 78–79)

Scenario. Regional offices now upload spreadsheets through the Mini Lab. Uploading is not enough — the files must be stored durably, validated automatically, cleaned on a schedule, and loaded into the warehouse so that dashboards are never more than a day stale. And the whole system must start on any machine with one command, because the next person to work on it is not you.

Explicit build steps:

1. Create the four MinIO buckets and document the naming convention. *(Week 19)*
2. Replace the CS-3 local-disk upload with a MinIO put; store bucket, key, size, checksum and version in Postgres. *(Week 19)*
3. Implement dataset versioning: uploading the same logical dataset creates a new immutable version; the API returns latest by default and a specific version on request. *(Week 19)*

4. Add an integrity check endpoint that reports metadata rows whose objects are missing. *(Week 19)*
5. Port the CS-1 validation rules into a declarative rule set, with each rule named and independently testable. *(Week 18)*
6. Write the extract task: pull the newest raw object from MinIO. *(Week 20)*
7. Write the validate task: apply the rule set; write passing rows onward and failing rows to processed-data/quarantine/ with reasons. *(Week 20)*
8. Write the clean task: deduplicate, normalise providers, standardise provinces and dates — reusing the CS-1 functions, now imported rather than rewritten. *(Week 20)*
9. Write the transform task: compute the derived fields the dashboards need, including completion rate by province. *(Week 20)*
10. Write the load task: upsert into the warehouse tables, idempotently. Re-running must not duplicate. *(Week 20)*
11. Assemble the five tasks into a DAG with a nightly schedule, two retries and exponential backoff; prove a mid-DAG failure resumes cleanly. *(Week 20)*
12. Write Dockerfiles for the frontend and backend; write docker-compose.yml covering frontend, backend, postgres and minio with healthchecks, named volumes and an .env.example. *(Week 21)*
13. Document the whole thing: a lineage diagram from CSV to dashboard, a runbook for docker compose up, and the data-stewardship statement. *(Week 21)*

Acceptance criteria: one docker compose up from a clean clone brings up all four services healthy; upload lands in MinIO with a metadata row; the DAG runs end to end and is provably idempotent; quarantined rows are retrievable with reasons; the lineage diagram traces one dashboard number back to a source file; the stewardship statement exists; all merged via reviewed PRs.

Forward links. The warehouse tables loaded here are the training data for CS-5's model. The Dockerfiles built here are the images pushed to Azure Container Registry in CS-5 and deployed by the CI/CD pipeline in CS-6. The quarantine mechanism becomes a monitored data-quality metric in Month 6.

E.11 Assessment, gate, badge, resources, facilitator notes (page 80)

Gate criteria (Month 4): can write a Dockerfile for FastAPI; can run the multi-service Compose stack; can store and retrieve an object from MinIO; can explain lineage from CSV upload to dashboard; can trigger or schedule a cleaning pipeline. **Badge: Pipeline Packager.**

Resources: Docker's own getting-started; Docker Compose file reference; MinIO quickstart and Python SDK docs; Airflow core concepts (DAGs, operators, scheduling) — concepts only, not deployment; Great Expectations core concepts; pandas IO and memory guidance; POPIA plain-language guides from the Information Regulator; Project Handbook Volumes II and V. 🌱 Extension: Polars user guide, DuckDB docs, Spark overview.

Facilitator notes. Docker is the single highest-variance topic in the programme: it works instantly for some learners and consumes days for others, usually for environment reasons entirely outside the curriculum (virtualisation disabled in BIOS, WSL2 not installed, corporate

antivirus, insufficient disk). **Run a Docker readiness check in Week 18, three weeks early**, so that machine-level problems surface with time to fix them. Have the low-resource path ready. Teach docker compose logs before anything else — a beginner who cannot read container logs is blind. On the pipeline: insist on idempotency being *demonstrated*, not claimed; run the DAG twice in front of the class and count the rows. On ethics: this is the month where the ethics content lands hardest, because learners now personally hold a dataset about people and have to decide how long to keep it.

Expected outcomes at the end of Month 4

The learner has an application that runs anywhere with one command, stores files properly, cleans data on a schedule without human intervention, and can prove where any number on its dashboard came from. They own a module in a shared codebase and have raised at least one issue with their own brief. They can explain to a non-technical person why a duplicate learner record is not a small problem.

---

12. Detailed Outline — Part F: Month 5, Analytics, ML & Azure (pages 81–94)

> **Theme:** Insights, predictions, and the cloud. **Project Handbook link:** Modules 7–8 and 14; Volumes II and V; FR-20 to FR-27, FR-32 to FR-33, FR-40. **Case study:** CS-5 *The Insight*. **Badge:** Cloud Analyst.

F.0 Month opener (page 81)

The Mini Lab currently stores and displays data. It does not yet *answer questions*. Month 5 closes that gap in two directions: descriptive (what happened — dashboards) and predictive (what is likely to happen — a dropout-risk model), and then puts the result somewhere other people can actually reach it (Azure). The opener is honest that one month is not enough to make anyone a data scientist, and states precisely what it *is* enough for: to build a defensible baseline model, evaluate it honestly, deploy it behind an API, and know the limits of what you have built.

F.1 Learning objectives (page 82)

By the end of Month 5, learners can: perform and report exploratory data analysis with appropriate descriptive statistics and visualisations *(C4-Independent)*; implement analytics endpoints and dashboard views for CHIETA KPIs *(C3-Independent, C4-Assisted)*; prepare features and labels and split data correctly *(C4-Assisted)*; train and evaluate classification models using accuracy, precision, recall, F1 and ROC-AUC, and explain why accuracy alone is misleading on imbalanced data *(C4-Independent)*; track experiments with MLflow and manage registry stages *(C4-Assisted)*; serve a model through POST /api/v1/predict *(C3-Independent, C4-Assisted)*; interpret feature importance and explain a prediction to a non-technical stakeholder *(C4-Assisted, C6-Independent)*; identify bias and fairness risks in a learner-risk model *(C6-Independent)*; and deploy a containerised service to Azure with secrets in Key Vault *(C5-Assisted)*.

F.2–F.11 Weekly breakdown (pages 83–91)

Week	Topics taught	🧪 Lab and 💥 break-it step	🏁 Deliverable

22	Descriptive statistics: mean, median, mode, spread, quartiles — and when the mean lies. Distributions; histograms; box plots; scatter plots; correlation matrices; correlation ≠ causation. Identifying trends, anomalies and relationships. Then the five CHIETA dashboards (Learner, Grant, ETQA, Skills, Executive): what question each answers, who reads it, what decision it drives. Two are implemented in full. Export basics (CSV/PDF) and scheduled-report concepts. 🌱 Superset connected to Postgres.	🧷 Produce a professional EDA report on the cleaned learner dataset and present the three most important findings in five minutes. 🧷 Build the Executive KPI API endpoint and its dashboard page. 💥 Compute a KPI two different ways (Python vs SQL), get different answers, and find out why.	An EDA report; two fully implemented dashboards on live data
23 ★	Supervised vs unsupervised learning. Features and labels. Feature engineering from the CHIETA data. Train/validation/test splitting and leakage. Logistic Regression and Random Forest with scikit-learn. Evaluation: confusion matrix, accuracy, precision, recall, F1, ROC-AUC; the class-imbalance trap. Baselines — always compare to "predict the majority class".	🧷 Train a dropout-risk classifier on the synthetic learner dataset; compare two algorithms against a baseline. 💥 Leak the label into a feature; observe 99% accuracy; find the leak. *(This lab is written out in full as Sample Page 2.)*	A trained, evaluated model with a written performance summary and an honest limitations section
24	MLflow: runs, parameters, metrics, artifacts; comparing experiments; the model registry and stages Development → Staging → Production; reproducibility. Then serving: loading a saved model in FastAPI, POST /api/v1/predict, POST /api/v1/models/train, input validation with Pydantic, and the Learner Success Prediction UI in Next.js.	🧷 Log five experiment runs with different hyperparameters and pick a winner from the MLflow UI, on evidence. 🧷 Save the model, load it in FastAPI, score new rows, render the result with a confidence indicator in the UI. 💥 Send a prediction request missing a feature; make the API return a clear 422 rather than a 500.	Working /predict endpoint and prediction UI; five tracked MLflow runs
—	**(Page 89)** Responsible and explainable AI: bias and fairness; where bias enters (historical data, sampling, proxy variables, labelling); feature importance; SHAP as an idea; communicating uncertainty; the difference between a prediction and a decision; who is accountable when a model is wrong. Applied directly: *should a CHIETA Analyst see an individual learner's dropout-risk score, and what might they do with it?*	🧷 Compute feature importance for the dropout model; identify any proxy for a protected attribute. 🧷 Structured debate on the deployment question, then write a 300-word position with a recommendation.	A model card: purpose, data, performance, limitations, fairness review, recommended use
25	Azure fundamentals: subscriptions, resource groups, regions, the portal and the CLI. Azure Container Registry: building, tagging and pushing images. Deploying containers with Azure Container Apps (or App Service) — AKS is *overviewed* only, and returns in Month 6. Azure Key Vault for the database URL and JWT secret; why secrets never enter a repository. Managed identity and connection strings as concepts. Cost awareness, budgets and alerts; basic	🧷 Push the CS-4 backend image to ACR; deploy it to Container Apps; hit a public HTTPS endpoint. 🧷 Move every secret into Key Vault and confirm the app still runs. 💥 Commit a secret to a branch on purpose, then practise the real remediation: rotate it, don't just delete the commit.	A publicly reachable HTTPS endpoint; zero secrets in the repository

	monitoring.		

F.11 Case Study CS-5 — *The Insight* (pages 92–93)

Scenario. CHIETA regional managers want two things. They want to see, at a glance, how each province and each provider is performing. And they want early warning: which currently-enrolled learners are at risk of not completing, while there is still time to intervene.

Explicit build steps:

1. Run EDA on the warehouse tables from CS-4; document distributions, null rates and outliers. *(Week 22)*
2. Define the five KPIs the Executive dashboard will show; write the SQL for each; validate each against a hand calculation. *(Week 22)*
3. Build the analytics endpoints returning those KPIs, paginated and filterable by province, provider and date range. *(Week 22)*
4. Build the Executive dashboard and the Learner dashboard in Next.js against those live endpoints, with all three data states handled. *(Week 22)*
5. Define the prediction problem precisely: *given a learner's first-eight-weeks record, predict whether they will complete.* Write down the label definition and the prediction window before touching data. *(Week 23)*
6. Engineer features: attendance rate, assessment submissions, days since last activity, provider historical completion rate, programme type, prior qualification level. *(Week 23)*
7. Split into train/validation/test by *enrolment cohort*, not randomly, to avoid temporal leakage. Explain why in writing. *(Week 23)*
8. Establish a baseline (majority class) and record its metrics. *(Week 23)*
9. Train Logistic Regression and Random Forest; evaluate both on precision, recall, F1 and ROC-AUC; choose one and justify the choice against the *use case*, not the highest number. *(Week 23)*
10. Track all runs in MLflow with parameters, metrics and artifacts; register the chosen model and move it to Staging. *(Week 24)*
11. Save the model; load it in FastAPI; implement POST /api/v1/predict with Pydantic input validation and a clear 422 on malformed input. *(Week 24)*
12. Build the Learner Success Prediction UI: select a learner, see a risk band (not a bare probability), see the top three contributing factors. *(Week 24)*
13. Compute feature importance; write the model card including a fairness review naming any proxy variables. *(Week 24)*
14. Containerise and push the backend image to ACR; deploy to Azure Container Apps; verify a public HTTPS endpoint. *(Week 25)*
15. Move all secrets to Key Vault; remove them from the repository and from .env files in version control; document the rotation procedure. *(Week 25)*

Acceptance criteria: two dashboards render live data with all states handled; every KPI matches an independent hand calculation; the model beats the baseline on recall for the at-risk class; five or more MLflow runs are logged and comparable; /predict returns a sensible response and a clean 422 on bad input; the prediction UI shows a band and contributing factors, never a bare

decimal; the model card is complete including limitations and fairness; at least the backend is reachable at a public HTTPS Azure URL; no secret appears anywhere in git history from this month forward.

Forward links. The deployed service is what the CI/CD pipeline in CS-6 will deploy automatically. The model card becomes part of the handover pack. The dashboards are the primary surface demonstrated at capstone.

F.12 Assessment, gate, badge, resources, facilitator notes (page 94)

Gate criteria (Month 5): can build a KPI dashboard from live SQL/API data; can train and evaluate a classification model and explain its metrics; can expose /predict from FastAPI; can push an image to ACR; can deploy a service to Azure and reach a public HTTPS endpoint. **Badge: Cloud Analyst.**

Resources: scikit-learn user guide (supervised learning, model evaluation, model selection); MLflow tracking and model-registry quickstarts; the Recharts gallery; Azure Container Apps quickstart, ACR quickstart, Key Vault concepts, Azure Cost Management basics; Google's *People + AI Guidebook* for communicating model output; Project Handbook Volumes II and V. 🌱 Extension: SHAP documentation, Superset, fairness-toolkit overviews.

Facilitator notes. The dominant risk this month is **accuracy worship**. Beginners fixate on a single number and will happily ship a model with 92% accuracy that never once predicts the at-risk class. Force the confusion matrix onto the screen before any accuracy figure is discussed, every single time. The second risk is **the leaky feature**: build the deliberate leak into the Week 23 lab so that every learner experiences suspiciously perfect performance and learns to distrust it. The third is **Azure cost anxiety** — set budgets and alerts on day one of Week 25, use the free tier, and give learners a documented teardown procedure so nobody is afraid to experiment. Finally: the ethics session must not be a lecture. Run it as a structured debate with learners assigned positions, because the argument is where the learning is.

Expected outcomes at the end of Month 5

The learner has an application that answers questions with data, makes a prediction, explains that prediction in terms a manager can act on, and is reachable on the public internet over HTTPS. They can also say clearly what their model should *not* be used for — which is the difference between someone who can train a model and someone who can be trusted with one.

---

13. Detailed Outline — Part G: Month 6, Quality, DevSecOps, Capstone & Handover (pages 95–108)

> **Theme:** Make it production-ready and teachable. **Project Handbook link:** Volumes V–VIII (testing, CI/CD, ops, governance); FR-13 to FR-15, FR-34 to FR-35, FR-38 to FR-40. **Case study:** CS-6 *The Product*. **Badge:** Data Lab Associate.

G.0 Month opener (page 95)

Stated as a rule, in bold, on the first line: **Month six is for finishing, not starting.** No new features. The temptation to add one more capability is the single most common reason capstones fail, and it is named here explicitly so facilitators can point at it in Week 27 when someone proposes a new dashboard. The work of this month is to take something that works on the builder's machine and turn it into something another person can run, trust, operate and extend. That transition — from *works* to *deliverable* — is the actual professional competency being certified.

G.1 Learning objectives (page 96)

By the end of Month 6, learners can: write automated unit, API and basic UI tests and explain the testing pyramid *(C6-Independent)*; implement data-quality checks in a pipeline *(C2-Independent)*; identify common web vulnerabilities from the OWASP Top 10 in their own code *(C5-Assisted, C6-Aware)*; build a CI/CD pipeline that tests on every pull request and deploys on merge *(C5-Independent)*; explain Kubernetes/AKS concepts and Helm at a working beginner level *(C5-Aware)*; operate the platform — read logs, respond to a P1–P4 incident, perform a backup and restore *(C5-Assisted)*; write documentation another developer can act on *(C6-Fluent)*; run a UAT session and obtain sign-off *(C6-Independent)*; and present a technical system to a non-technical audience *(C6-Fluent)*.

G.2–G.9 Weekly breakdown (pages 97–105)

Week	Topics taught	Lab and break-it step	Deliverable
26	The testing pyramid: many unit, fewer integration, few end-to-end, and why the inverse is a trap. What to test and what not to. pytest for FastAPI: fixtures, test client, database fixtures, mocking external calls. Vitest and Testing Library for components. Playwright for critical-path UI smoke tests. Data-quality checks as tests on the pipeline. Then security: the OWASP Top 10 in plain language with the four that actually matter here — broken access control, injection, security misconfiguration, vulnerable dependencies. Dependency scanning.	Write 10+ API tests including auth and RBAC paths. Write 3 UI smoke tests: login, upload, dashboard loads. Add a data-quality assertion that fails the DAG on a null-rate spike. Remove a role check and confirm a test catches it.	≥10 passing API tests, 3 UI smoke tests, 1 pipeline quality gate
27	CI/CD: what a pipeline is; stages build → test → docker build → push to ACR → deploy. Azure DevOps Pipelines or GitHub Actions YAML. Triggering on pull request vs on merge. Branch protection. The branching model in practice (main, develop, feature/*). The code-review checklist. Versioning and release notes.	Write the pipeline YAML; open a PR with a failing test and watch the pipeline block the merge. Merge and watch the deploy run. Break the build deliberately and read the pipeline log to find the failing step.	A pipeline that runs tests on every PR and deploys on merge to main
28	Kubernetes and AKS: pods, deployments, services, namespaces (the Project Handbook's namespace layout), why an orchestrator exists. Helm charts conceptually. Then observability: what to log and what never to log; structured logs;	Deploy the Compose stack or a simplified Container Apps/AKS deployment; add health monitoring. Perform a **full backup and restore drill**: take a backup, destroy the database, restore it, prove no data	A monitored deployment; a completed and documented restore drill

	metrics; Prometheus and Grafana concepts; health and readiness endpoints; alerting thresholds. Incident management: severities P1–P4 and the Project Handbook SLA; the incident log. Backup and restore for PostgreSQL.	loss. 💥 Kill a running container and observe what the health check and the user both see.	
—	**(Page 103) Consolidation Week 2 — feature freeze.** Monday: freeze; no new features from this point. Tuesday–Wednesday: full-system bug sweep — every learner tests every other learner's module and logs findings in a shared bug register; triage into must-fix / should-fix / won't-fix with reasons. Thursday: fix the must-fixes. Friday: first full capstone rehearsal with peer feedback and timing.	💉 Cross-testing rota; bug register; triage meeting run by the learners with the facilitator observing only.	A triaged bug register with must-fixes closed; a timed rehearsal completed
29	UAT: writing a test script from user stories; acceptance criteria; recruiting a non-technical tester; recording results; the sign-off sheet. Documentation: the README, the architecture diagram, the API reference, the Admin user guide, the Analyst user guide, the developer guide for the module you own, and the runbook. Training materials: a 10-minute platform walkthrough script and recording.	💉 Run UAT with a non-technical person (another learner from a different role, a facilitator, or an invited CHIETA stakeholder) executing the script unaided while the builder watches in silence — **the builder may not intervene.** 💥 Watch a user fail a task you were sure was obvious; fix the interface, not the user.	Executed UAT script with results and sign-off; complete documentation pack
30	Capstone demo day and technical interview. Handover. Contribution report. Reference letter. Programme retrospective and career conversation.	Live demonstration to a panel; 20-minute technical interview from the Appendix F question bank; formal handover of the repository and documentation.	Capstone delivered; handover pack accepted; Associate badge issued

G.10 Case Study CS-6 — *The Product* (pages 106–107)

Scenario. The Mini Lab works. Now prove it: prove it is correct (tests), prove it can be changed safely (CI/CD), prove it can be operated (monitoring, backup, incidents), prove it can be used (UAT), and prove it can be inherited (documentation and handover).

Explicit build steps:

1. Write the test plan: what is covered, what is not, and why. *(Week 26)*
2. Unit tests for the cleaning and transform functions — the CS-1 descendants. *(Week 26)*
3. API tests for auth, RBAC, dataset CRUD, analytics and predict — including the negative cases (403, 401, 422). *(Week 26)*
4. Component tests for the KPI card, filter bar and prediction UI. *(Week 26)*
5. Three Playwright smoke tests on the critical paths: login, upload, dashboard render. *(Week 26)*
6. Add a data-quality gate to the DAG that fails the run on a null-rate or row-count anomaly. *(Week 26)*
7. Run a dependency scan; upgrade or document every finding. *(Week 26)*

8. Write the CI pipeline: install → lint → test → build image → push to ACR. Trigger on pull request. *(Week 27)*
9. Add the CD stage: deploy to Azure on merge to main. Enable branch protection so tests must pass. *(Week 27)*
10. Add health and readiness endpoints; wire basic monitoring and one alert. *(Week 28)*
11. Perform and document the PostgreSQL backup and restore drill, with timings. *(Week 28)*
12. Write the incident runbook: severities, first responses, escalation, and the three most likely failures in *this* system. *(Week 28)*
13. Feature freeze; cross-test; triage; close must-fix bugs. *(Consolidation Week 2)*
14. Write the documentation pack: README, architecture diagram, API reference, Admin guide, Analyst guide, developer guide, runbook. *(Week 29)*
15. Write and execute the UAT script with a non-technical tester; record results; obtain sign-off. *(Week 29)*
16. Deliver the capstone demo and technical interview; hand over the repository, documentation and Volume VII-style handover checklist. *(Week 30)*

The nine capstone requirements — all must work:

1. A Next.js UI with working RBAC across all four roles
2. A FastAPI /api/v1 service with accurate Swagger documentation
3. PostgreSQL for structured data and MinIO for objects
4. docker compose up running the full stack locally from a clean clone
5. At least one service deployed and reachable on Azure over public HTTPS
6. One ETL job and one ML prediction feature, both working
7. Automated tests running in CI on every pull request
8. README, architecture diagram, and user guides for Admin and Analyst
9. A completed handover checklist in the Project Handbook Volume VII style

G.11 Final assessment, badge and exit (page 108)

The capstone demo (30 minutes): a 15-minute live demonstration walking the end-to-end user journey — Admin creates an Analyst; Analyst uploads a dataset; the pipeline cleans it; the dashboard updates; the model predicts risk; the audit log shows every action — followed by a 5-minute walk-through of one piece of code the learner is proud of, with an explanation of *why* it was written that way, and 10 minutes of questions. Presented to a panel of at least two, including someone non-technical.

The technical interview (20 minutes): drawn from the Appendix F bank. Questions are about the learner's *own* system: *why did you choose that join?*, *what happens if MinIO is down when someone uploads?*, *how would you add a fifth role?*, *what would break first if this had a hundred times more data?*, *what should this model never be used for?* The interview tests understanding, not recall.

Required for the Data Lab Associate badge: all nine capstone requirements demonstrated; capstone demo passed; technical interview passed; handover pack accepted; all four phase gates met; contribution log complete.

Exit artefacts issued to every completer: a written reference letter from the lead facilitator; a contribution report listing every merged PR with reviewer names; the Data Lab Associate digital badge (FR-38/FR-39); and written permission to describe the project and the module owned on a CV and public portfolio.

Expected outcomes at the end of Month 6

The learner can build, test, deploy, operate, document and hand over a data platform, and can explain any part of it to either an engineer or a manager. Six months earlier they had never opened a terminal. That is the whole claim of this programme, and CS-6 is the evidence for it.

---

14. Detailed Outline — Parts H, I, J (pages 109–128)

Part H — The Assessment System (pages 109–115)

- **Page 109 — Philosophy.** Why there are no written exams, in full: the competency is building and explaining a working system; written exams measure recall, correlate with prior educational advantage, and would undermine the access purpose of a SETA-funded programme. What is assessed instead: merged code, weekly deliverables, live demonstration, and spoken or written explanation of one's own decisions.

- **Page 110 — The weighting model**, adapted from the Intern Training Plan:

Component	Weight	Evidence
Code quality and contributions	30%	Merged PRs, review feedback, adherence to standards
Weekly deliverables	25%	The 24 weekly artefacts, on time and working
Monthly case study (CS-1 to CS-5)	20%	Demo against published acceptance criteria
Documentation	10%	Guides, README, code comments, model card
Collaboration	8%	Quality of help requests, peer reviews, stand-up participation
Capstone and interview	7%	Demo clarity, code explanation, reflection

  With the two documented departures from the source model (weekly deliverables raised from 20% to 25%; no marks attached to the entry diagnostic) and the rationale for each.

- **Page 111 — Formative instruments.** The Friday exit ticket (three questions, five minutes, never graded — used to plan Monday); the Friday five-minute demo; the code review; the Monday stand-up self-report; the mid-month peer walkthrough. Guidance on *feedback latency*: formative feedback older than 48 hours is decoration.

- **Page 112 — Summative instruments.** The monthly case-study demo; the 400-word reflection; the skills checklist; the phase gate.

- **Page 113 — The four gates**, their criteria, and the consequence of missing each — including the deliberate design decision that only the Month 3 gate is genuinely blocking, and why the others are remediated in parallel rather than by delay.

- **Page 114 — The capstone rubric** (five criteria × four levels) and the technical-interview question bank organised by competency domain.

- **Page 115 — Moderation, appeals, accommodation, records.** Second-marking of all capstone demos; the appeal route; reasonable accommodation for disability and for learners with caregiving or employment constraints; what evidence is retained, for how long, and where — including the POPIA implications of retaining learner performance records.

Part I — Facilitator Playbook (pages 116–122)

- **Page 116 — The facilitator's week.** Preparation checklist (environment tested on a clean machine before every lab; every lab completed personally by the facilitator before it is taught — non-negotiable); session opening script (recap → today's question → why it matters → demo) and closing script (what we built → the one exercise → what breaks if you skip it).
- **Page 117 — Running a type-along lab with mixed pace.** The three-speed problem and its solutions: checkpoint commits learners can git checkout to rejoin; a visible "we are here" marker; the fast-finisher 🌱 task always ready; the rule that the facilitator types *slower* than feels natural and narrates every keystroke that is not obvious.
- **Page 118 — Twenty beginner misconceptions**, with the pre-emptive framing for each. Examples: a variable is a box, not a label; = assigns, == compares; a function returns, it does not necessarily print; a for over a dict yields keys; null and undefined are different absences; the frontend and backend are two programs, not one; a container is not a virtual machine; a commit is local until pushed; a merged PR is not a deploy; higher accuracy is not a better model.
- **Page 119 — The four-question triage** for a stuck learner: *What were you trying to do? What did you expect? What actually happened — read me the first line? What have you already tried?* With explicit guidance never to take the keyboard, and the escalation rule for when a learner is stuck on an *environment* problem rather than a *learning* problem (take the keyboard immediately; environment problems teach nothing).
- **Page 120 — Code review as teaching.** How to write a review comment a beginner can act on: name the specific line, state the concern, state the consequence, propose a direction, do not rewrite it for them. The distinction between blocking and non-blocking comments. Reviewing tone in a cohort where the reviewer is also the assessor.
- **Page 121 — The monthly one-on-one.** Structure, the four standing questions from the Intern Training Plan, and the boundary: the facilitator is not a counsellor. Referral routes for learners in financial, housing or mental-health difficulty, which in a SETA cohort are common and are the leading non-academic cause of dropout.
- **Page 122 — Risk register.** The eight programme risks with early-warning signs and interventions: hardware failure; connectivity; learner attrition in Weeks 3–4; the Month 3 bottleneck; Docker environment failures; Azure cost overrun; facilitator single-point-of-failure; and scope creep in Month 6.

Part J — Learner Support, Progression & Professional Practice (pages 123–128)

- **Page 123 — The support ladder.** Level 1 self-help (always first: read the error, check the codebase, search, check the docs); Level 2 peer after 30 minutes; Level 3 mentor after 2 hours or on a hard blocker (response within 4 working hours); Level 4 session escalation for architectural questions, raised at Monday stand-up. The four-part help

request format, with a worked good example and a worked bad example side by side. Stated plainly: writing a good help request is a graded professional skill, not a courtesy.

- **Page 124 — The contribution log.** Fields: week, task in one or two sentences, hours, PR link or commit hash, review outcome, reviewer name. Who keeps it (the learner), who checks it (the facilitator), and what it becomes (the reference letter and contribution report). Worked example of a strong entry.
- **Page 125 — Study skills for beginners.** Spaced retrieval over re-reading; the rebuild-from-memory step in the five-step loop; deliberate failure; how to take notes on code (write the *why*, the *what* is in the file); how to read documentation without drowning; managing the feeling of not understanding, which is the normal condition of learning to program and not a signal to stop.
- **Page 126 — Working in a team codebase.** Branch naming; commit message conventions; PR description template; how to receive a review comment without taking it personally; how to disagree with a reviewer productively; what to do when you break main.
- **Page 127 — Ethics, POPIA and professional conduct.** Consolidates the ethics thread: purpose limitation, minimality, retention, security, data-subject rights; the difference between de-identified and anonymous; what to do if you encounter real personal data in a training environment; the professional obligation to raise a concern about a system you are building.
- **Page 128 — After the programme.** The three realistic next steps (junior developer, junior data engineer, junior analyst) with the additional competencies each needs; how to describe this programme on a CV; how to present the Mini Lab as a portfolio piece; the WIL and further-study routes; and the CHIETA student-contributor path.

---

15. Detailed Outline — Part K: Appendices (pages 129–147)

Appendix	Pages	Content
A — Synthetic dataset family and data dictionary	129–131	Seven CSVs designed for chemical-sector skills data: learners.csv (learner_id, pseudonymous name, province, programme, prior_qualification, enrolment_date, status), enrolments.csv, providers.csv, qualifications.csv, assessments.csv, workplace_placements.csv, grants.csv. Full data dictionary: field, type, nullable, valid values, known defects. **The known-defects table is the pedagogically important part** — it lists exactly which rows are duplicated, which dates are impossible, and which provider names vary, so facilitators can verify learner solutions deterministically. Plus the generator script so cohorts can have fresh data.
B — Software install list	132	Every tool, version, purpose and source URL; VS Code extension list; Python and Node package lists; the low-resource substitution table.
C — 26-week master schedule	133	One table: week, month, case study, topics, lab, deliverable, gate, badge. The single reference page for programme managers.
D — Week 0 diagnostic	134	The five tasks, administration instructions, scoring bands, and the routing decision table. Explicitly labelled non-exclusionary.
E — Competency descriptors and matrix	135–137	C1–C6 at four levels with observable behaviours; the matrix mapping every assessment instrument to the competencies it evidences. This is the audit artefact for CHIETA.
F — Rubrics and	138–	Code quality; documentation; demo; peer review; reflection; capstone; the technical-

question bank	139	interview question bank grouped by domain with model answers for facilitators.
G — Templates	140	Task breakdown; contribution log; help request; peer review; PR description; incident log; data-stewardship statement; model card.
H — FR coverage map	141	FR-1 to FR-40 against the week each is introduced and the week each is implemented in the Mini Lab, extending the Beginner Course's Appendix A.
I — Project Handbook volume mapping	142	Volumes I–VIII against months, showing which volume sections are used as reference in which week — and which are explicitly *not* given to learners early.
J — Module briefs	143	The five role-based module briefs (frontend, UI/UX, data/backend, full-stack, QA/documentation) from the Intern Training Plan, each expanded into a written brief with scope, screens, data, API surface, and acceptance criteria.
K — UAT script and sign-off	144	The end-to-end user journey as a numbered test script with expected results, pass/fail columns, and a sign-off block.
L — Handover checklist	145	Volume VII-style: repository access, documentation, credentials rotation, deployment details, known issues, support contacts, outstanding work.
M — Glossary	146	Full glossary, ~150 terms, extending the page-8 starter set.
N — Extension menu and further reading	147	The 🌱 track: Superset, Spark, full AKS with Helm, OpenMetadata, Polars/DuckDB, advanced testing, SHAP, performance profiling. Curated further reading per domain, free sources prioritised.

---

16. Two Fully Written Sample Pages

The two pages below are reproduced at the depth, tone and structure every page of the assembled handbook should meet. They are drawn from different modules and different months so that the format can be seen working at both ends of the difficulty range: one for a learner in Week 3 who has been programming for a fortnight, and one for a learner in Week 23 who is training their first model.

★ SAMPLE PAGE 1 — Handbook page 30

B.5 Week 3, Session 1 — Functions: Teaching a Program to Remember How

Month 1 · Foundations · Competency C1 · 90-minute taught session + 2-hour lab
Prerequisite: Week 2 (conditions, loops, tracing code) · **Feeds:** CS-1 steps 4–8

🎯 What you will be able to do by the end of this session

1. Write a function that takes parameters and returns a value.
2. Explain the difference between a function that *prints* and a function that *returns* — and say why the difference matters.
3. Take a task described in a sentence and break it into two or three functions before writing any code.
4. Spot repeated code and pull it into a function.

Concept: why functions exist

Last week you wrote a grade calculator. If CHIETA asked you to calculate grades in three different places in the same program, you had two options: copy the code three times, or find some way to write it once and use it three times. Copying works right up until the moment the rules change — and then you have to remember all three places, and you will miss one. That missed copy is a bug, and it is a bug nobody notices until someone's certificate is printed with the wrong result.

A **function** is a named piece of code that does one job. You write it once, give it a name, and call it whenever you need that job done. It can take information in (**parameters**) and it can hand information back (**return value**).

> **Why this matters for the Data Lab.** By Week 5 you will have written a function called completion_rate_by_province(). In Month 3 that same function becomes an API endpoint that a browser can call. In Month 5 it becomes an input to a machine-learning model. The function you write on Thursday of this week is genuinely the same code, still running, five months from now. This is not a practice exercise.

Session plan (90 minutes)

Time	Activity	Facilitator does	Learners do
0–10	**Recap and the problem**	Show last week's calculator with the grading logic copy-pasted three times. Change one rule. Miss one copy on purpose. Run it. Get a wrong answer.	Watch. Spot the missed copy.
10–25	**Concept: def, parameters, return**	Live-code calculate_grade(mark) on screen. Narrate every keystroke. Call it three times.	Type along.
25–40	**The print/return trap**	Write two versions side by side: one that prints, one that returns. Try to use each one inside another calculation. Show that the printing version returns None.	Type along. Predict which one breaks *before* running it.
40–55	**Scope**	Show a global variable being modified inside a function. Introduce a bug that depends on call order.	Trace the bug on paper before running.
55–70	**Decomposition on paper**	Give the task: *"read a list of learner records and report the completion rate per province."* Refuse to let anyone open an editor. Build the function list on the whiteboard as a class.	Write the function names and their inputs/outputs on paper.
70–85	**Guided exercise**	Circulate. Use the four-question triage. Do not take keyboards.	Write calculate_grade() and format_result(), and call both from a main().
85–90	**Break it and close**	Instruct: shadow a variable inside a function so it silently does the wrong thing.	Break it, find it with print-tracing, fix it. Submit the exit ticket.

🔧 Lab exercise (2 hours, follows the session)

Task. You are given messy_grades.py — a working 60-line script that does the right thing in the wrong way. It repeats the same grading logic four times and uses three global variables.

Steps.

1. **Read it and do not change anything yet.** Write, on paper, what each block does in one sentence.
2. Identify the repetition. Circle the four places the same logic appears.
3. Write calculate_grade(mark) that **returns** a symbol. Do not print inside it.
4. Replace all four repetitions with calls to your function. Run the script. The output must be identical to before — this is your safety net.
5. Find the three globals. Convert each into a parameter passed in and a value returned out.
6. Write format_learner_line(name, mark, symbol) that returns a display string.
7. Add a docstring to every function: one line on what it does, one line per parameter, one line on what it returns.
8. 💥 **Break it on purpose:** inside calculate_grade, add a line mark = 0 at the top. Run it. Every learner now gets an F. Explain out loud why this happened, then remove it.
9. Commit on a branch named week3/functions-refactor and open a pull request.

Definition of done: the script produces identical output to the original; there is no repeated grading logic; there are no global variables; every function has a docstring; the PR is open and reviewed.

🌱 **Extension:** add a grade_all(learners) function that takes a list of dictionaries and returns a list of formatted lines, so main() becomes three lines long.

⚠ Common errors this week — and what they mean

What you see	What it usually means	What to do
TypeError: calculate_grade() missing 1 required positional argument: 'mark'	You called the function without giving it the information it needs.	Check the brackets: calculate_grade() should be calculate_grade(78).
Your function prints the right thing but the variable holding it is None	The function prints instead of returning.	Add a return statement. print shows a human; return tells the program.
IndentationError: expected an indented block	The lines that belong inside the function are not indented under def.	Indent every line of the body by four spaces.
NameError: name 'symbol' is not defined outside the function	You are trying to use a variable that only exists inside the function.	Return it, and store the result: symbol = calculate_grade(78).
The function works alone but gives wrong answers when called in a loop	A global is being modified and carried between calls.	Pass what it needs in; return what it produces out. Nothing shared.

☑ Checkpoint — exit ticket (5 minutes, not graded)

1. In one sentence, what is the difference between print and return?
2. Rewrite this in your own words: *a function should do one job.* Give an example of a function doing two jobs.
3. What confused you most today?

Facilitators: read all exit tickets before Monday. If more than a third of the cohort names the same confusion, reteach it in Monday's recap. Do not carry an unresolved confusion into Week 4 — dictionaries and nested structures assume functions are solid.

📎 Deliverable due before Thursday's session

A merged pull request containing the refactored script, with the decomposition plan you wrote on paper photographed and committed to /docs/week3-plan.jpg. **The paper plan is assessed.** Code that arrives without evidence of planning is marked incomplete even if it runs — planning before coding is the habit being built here, and it is the habit that makes Month 4 survivable.

★ SAMPLE PAGE 2 — Handbook page 86

F.5 Week 23, Session 2 — Training and Evaluating the Dropout-Risk Model

Month 5 · Analytics & AI · Competency C4 · 90-minute taught session + 2-hour lab
Prerequisite: Week 22 (EDA), Week 23 Session 1 (features, labels, splitting) · **Feeds:** CS-5 steps 8–10, and the model card in Week 24

🎯 What you will be able to do by the end of this session

1. Establish a baseline and explain why a model that cannot beat it is worthless.
2. Train a Logistic Regression and a Random Forest classifier with scikit-learn.
3. Read a confusion matrix and compute precision, recall and F1 from it by hand.
4. Choose between two models on the basis of the *use case*, not the highest number.
5. Recognise data leakage from the symptom — performance that is too good.

💡 Concept: accuracy is the number that lies to you

Suppose 88% of CHIETA learners complete their programme. A "model" consisting of a single line — return "will complete" — is 88% accurate. It has learned nothing. It will never identify a single at-risk learner. It is completely useless for the thing CHIETA actually wants, which is early warning.

This is why you always establish a **baseline** first and why you never report accuracy alone.

For this problem, the class we care about is the *minority* class: learners at risk of not completing. Two numbers matter:

- **Recall** (of the at-risk class): of all the learners who genuinely will not complete, what fraction did we flag? Low recall means we miss people who needed help.
- **Precision** (of the at-risk class): of all the learners we flagged, what fraction genuinely will not complete? Low precision means we waste scarce intervention resources — and we label learners as at-risk when they are not, which has a real human cost.

The trade-off is a policy decision, not a technical one. If CHIETA has capacity to intervene with 200 learners, precision matters more. If the cost of missing a learner is much higher than the cost of an unnecessary check-in, recall matters more. Your job as the engineer is to make the trade-off *visible* to the person who is entitled to make it — not to make it silently by picking whichever model scored highest.

Session plan (90 minutes)

Time	Activity	Facilitator does	Learners do
0–10	**The 88% model**	Write the one-line "model". Compute its accuracy live. Ask the class whether CHIETA should deploy it.	Argue. Discover the problem themselves.
10–25	**The confusion matrix**	Draw the 2×2 on the board with real counts. Derive precision, recall and F1 from it by hand, no library.	Compute the three metrics on paper from a second matrix.
25–45	**Training two models**	Live-code the scikit-learn pipeline: fit, predict, classification_report, confusion_matrix. Train Logistic Regression, then Random Forest.	Type along on the shared dataset.
45–60	**Reading the results together**	Put both confusion matrices side by side. Ask which model to ship, and for whom. Refuse to accept "the one with higher accuracy" as an answer.	Defend a choice with reference to the use case.
60–80	💥 **The planted leak**	Instruct learners to add weeks_to_completion as a feature. Accuracy jumps to ~0.99. Let the room celebrate. Then ask: *when would we know that value in real life?*	Experience the false triumph. Work out that the feature encodes the answer.
80–90	**Close**	Formalise leakage: any feature only knowable after the outcome. Name the rule: *if it is suspiciously good, it is suspicious.*	Exit ticket.

Lab exercise (2 hours)

Dataset: learner_features.csv from your own CS-4 warehouse — the data your own pipeline produced. **Label:** completed (0/1). **Prediction window:** first eight weeks of enrolment only.

Steps.

1. Load the data. Confirm the split you created in Session 1 is by **enrolment cohort**, not random. Write one sentence in your notebook explaining why random splitting would leak.
2. Compute and record the baseline: predict the majority class for everything. Record its accuracy, precision, recall and F1 for the at-risk class. **You will compare everything to this.**
3. Train a Logistic Regression. Record all four metrics plus ROC-AUC on the validation set.

4. Train a Random Forest with default settings. Record the same five numbers.
5. Print both confusion matrices. For each, state in writing how many at-risk learners were missed and how many non-at-risk learners were incorrectly flagged.
6. Vary one hyperparameter on the Random Forest (max_depth, three values). Record each result. You will log these in MLflow next week — start keeping the numbers now.
7. Choose one model. Write a short paragraph justifying it in terms of what CHIETA would do with the output. A justification that only cites a metric is not accepted.
8. 💥 **Reproduce the leak deliberately:** add weeks_to_completion as a feature, retrain, record the result, then remove it and write two sentences on how you would catch a leak like this in future.
9. Write the limitations section: what this model does not know, who it may work worse for, and what it must never be used for.
10. Commit the notebook and a model_results.md table on branch week23/dropout-model; open a PR.

Definition of done: the baseline is recorded; both models beat it on recall for the at-risk class, or you have explained in writing why they do not; both confusion matrices are in the PR; the model choice is justified against the use case; the leakage experiment is documented; limitations are written.

🌱 **Extension:** add class weighting or resampling and report the effect on precision and recall; compute a precision-recall curve and identify the operating point that flags exactly 200 learners.

⚠ Common errors this week — and what they mean

What you see	What it usually means	What to do
Accuracy above 0.97 on the first attempt	Leakage. A feature encodes the outcome.	List every feature and ask, for each: *would we know this in week 8?* Drop the ones that fail.
Recall for the at-risk class is 0.00	The model predicts the majority class for everything. It is the baseline wearing a costume.	Check class balance; apply class weighting; question whether your features carry any signal at all.
ValueError: could not convert string to float	A categorical column (province, provider) was passed in raw.	Encode categoricals before fitting.
Validation results far worse than training results	Overfitting, most often an unconstrained Random Forest.	Constrain depth or minimum samples per leaf; report both numbers honestly rather than hiding the gap.
Results change on every run	No random seed set.	Set random_state. Reproducibility is not optional in work anyone will rely on.

☑ Checkpoint — exit ticket (5 minutes, not graded)

1. A model has 91% accuracy and 0.04 recall on the at-risk class. Should CHIETA deploy it? One sentence.
2. Name one feature in this dataset that would be leakage, and say why.
3. What confused you most today?

Deliverable due before Week 24

A merged PR containing the notebook, model_results.md with all recorded runs including the baseline and the leakage experiment, and the written limitations section. **The limitations section is assessed as heavily as the model.** A learner who trains a strong model and cannot say what it should not be used for has not met this week's outcome.

> **Facilitator note.** Do not rescue the room during the leakage step. The celebration when accuracy hits 0.99, followed by the realisation two minutes later, is the most durable teaching moment in Month 5. Let it happen at full volume. Learners who experience it remember it; learners who are merely warned about it do not.

---

17. Production, Editing and Format Notes

17.1 What has been delivered and what remains

This document is the **master blueprint**: the reasoning, the full architecture, the page-by-page outline of all 147 pages, and two pages written out in full to fix the standard for the rest. Writing the remaining pages to this depth is a content-production task of roughly 55,000–70,000 words. The blueprint is deliberately specific enough that it can be split across several writers without the result reading as though it was.

Recommended production sequence, if the handbook is to be assembled by a team:

1. **Appendix A (the synthetic datasets) first.** Nothing else can be written accurately until the data exists, because every lab, every acceptance criterion and every worked example references specific fields and specific known defects. Generate the data, then freeze it.
2. **Parts B and C next** (Months 1–2). These carry the highest risk: beginner content is the hardest to write well, and errors here compound through the whole programme.
3. **Part A and the front matter third**, once the teaching content exists to describe accurately.
4. **Parts D–G**, which can be written in parallel by different authors because the case-study briefs in this blueprint fix the interfaces between months.
5. **Parts H, I, J and the remaining appendices last**, as they are largely assembled from the rubrics and templates already specified.

Assign one **consistency editor** across the whole document. The single greatest quality risk in a multi-author handbook is drift in the assumed level: Month 4 written as if for people who already know Docker, when the reader is a person who first opened a terminal nineteen weeks ago.

17.2 Editing the master file

The deliverable is Markdown, chosen for exactly the reasons the brief implies: it is plain text, so it diffs and merges in Git; it converts cleanly to Word, PDF and HTML; and no instructor is locked out by not owning particular software.

Recommended editing workflow. Keep the handbook in the same Git repository as the courseware, under /docs/handbook/. Split the single master file into one file per Part (00-front-matter.md, 01-part-a.md, … 11-appendices.md) once production begins — an 80+ page single file becomes unwieldy for concurrent editing. Instructors propose changes by pull request, exactly as learners do. This is not administrative pedantry: it means the handbook is maintained using the same workflow the handbook teaches, and every cohort's improvements are preserved with attribution.

Version discipline. Increment the minor version each cohort (1.0 → 1.1) and record what changed and why, sourced from the Friday retrospectives and the Consolidation Week feedback. A handbook that is not revised after its first cohort has not been used.

17.3 Conversion

Target	Command	Use
Word (instructor editing)	pandoc handbook.md -o handbook.docx --toc --toc-depth=3 --reference-doc=cxi-template.docx	Instructors who prefer tracked changes; CHIETA review cycles
PDF (distribution)	pandoc handbook.md -o handbook.pdf --toc --pdf-engine=xelatex	Printed learner copies, formal submission
HTML (learner-facing)	pandoc handbook.md -o index.html --toc --standalone --css=cxi.css	The searchable learner portal; the version learners will actually use

Build a cxi-template.docx carrying CXi Africa and CHIETA branding, heading styles and page furniture, and pass it via --reference-doc so that branding is applied at conversion rather than maintained by hand in the source. The A3 wall chart (page 17) and the lineage diagrams should be authored as separate SVG assets and referenced, not drawn in Markdown.

17.4 The objectives this handbook must keep meeting

Every future edit should be tested against the four commitments the design rests on. If a change weakens one of them, it is the wrong change.

1. **A complete beginner must be able to follow it.** The test is not whether the content is correct; it is whether a person who first opened a terminal in Week 0 can act on the page without external help. Any page that assumes unstated prior knowledge has failed, however accurate it is.
2. **Every week must ship something visible.** If an edit produces a week whose deliverable cannot be seen, run or clicked, the week has been broken. Restructure it rather than accepting an invisible week.
3. **The spine must stay unbroken.** CS-1 through CS-6 form one artefact. If a change to Month 4 means the Month 5 model can no longer be trained on the Month 4 warehouse, the change has severed the spine and the whole pedagogical argument of the programme collapses into a set of disconnected tutorials.
4. **Assessment must certify building and explaining, not recalling.** Any drift toward written tests of terminology should be reversed. The learner is being certified to build a system and to account for it — and the assessment must produce evidence of exactly that, in a form CHIETA can audit.

Prepared for CXi Africa · CHIETA Data Lab Training Programme · Master Blueprint v1.0 Built on: Lesson_plan.docx, CHIETA_Data_Lab_6_Month_Beginner_Course.docx, and CHIETA_DataLab_Intern_Training_Plan.docx (Lethabo Molefe).